

gender of the individual. Always consult a physician or qualified health care professional before beginning or modifying an exercise program.] Write an app that inputs the person's age, then calculates and displays the person's maximum heart rate and target-heart-rate range.

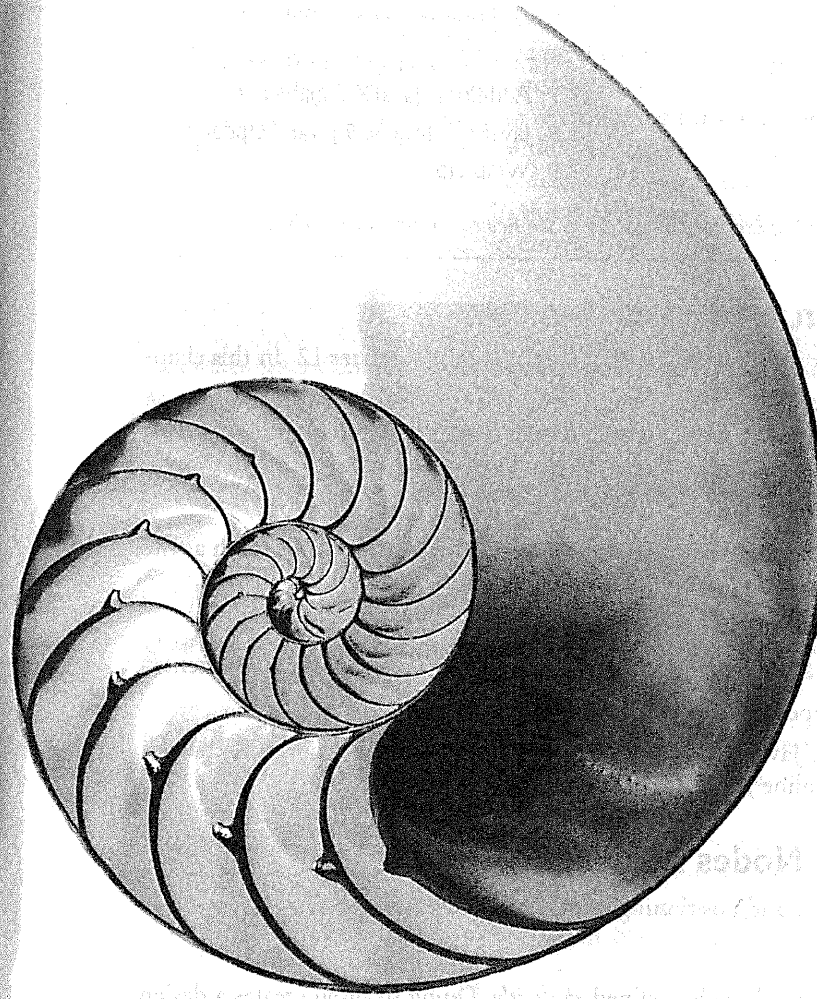
JavaFX GUI: Part 2

13

Objectives

In this chapter you'll:

- Learn more details of laying out nodes in a scene graph with JavaFX layout panels.
- Continue building JavaFX GUIs with Scene Builder.
- Create and manipulate **RadioButtons** and **ListViews**.
- Use **BorderPanes** and **TitledPanes** to layout controls.
- Handle mouse events.
- Use property binding and property listeners to perform tasks when a control's property value changes.
- Programmatically create layouts and controls.
- Customize a **ListView**'s cells with a custom cell factory.
- See an overview of other JavaFX capabilities.
- Be introduced to the JavaFX 9 updates in Java SE 9.



- 13.1 Introduction
- 13.2 Laying Out Nodes in a Scene Graph
- 13.3 Painter App: RadioButtons, Mouse Events and Shapes
 - 13.3.1 Technologies Overview
 - 13.3.2 Creating the Painter.fxml File
 - 13.3.3 Building the GUI
 - 13.3.4 Painter Subclass of Application
 - 13.3.5 PainterController Class
- 13.4 Color Chooser App: Property Bindings and Property Listeners
 - 13.4.1 Technologies Overview
 - 13.4.2 Building the GUI
 - 13.4.3 ColorChooser Subclass of Application
 - 13.4.4 ColorChooserController Class
- 13.5 Cover Viewer App: Data-Driven GUIs with JavaFX Collections
 - 13.5.1 Technologies Overview
 - 13.5.2 Adding Images to the App's Folder
 - 13.5.3 Building the GUI
 - 13.5.4 CoverViewer Subclass of Application
 - 13.5.5 CoverViewerController Class
- 13.6 Cover Viewer App: Customizing ListView Cells
 - 13.6.1 Technologies Overview
 - 13.6.2 Copying the CoverViewer App
 - 13.6.3 ImageTableCell Custom Cell Factory Class
 - 13.6.4 CoverViewerController Class
- 13.7 Additional JavaFX Capabilities
- 13.8 JavaFX 9: Java SE 9 JavaFX Updates
- 13.9 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises

13.1 Introduction

This chapter continues our JavaFX presentation¹ that began in Chapter 12. In this chapter, you'll:

- Use additional layouts (TitledPane, BorderPane and Pane) and controls (RadioButton and ListView).
- Handle mouse and RadioButton events.
- Set up event handlers that respond to property changes on controls (such as the value of a Slider).
- Display Rectangles and Circles as nodes in the scene graph.
- Bind a collection of objects to a ListView that displays the collection's contents.
- Customize the appearance of a ListView's cells.

Finally, we overview other JavaFX capabilities and mention Java SE 9's JavaFX changes that are discussed in our online Java SE 9 chapters.

13.2 Laying Out Nodes in a Scene Graph

A layout determines the size and positioning of nodes in the scene graph.

Node Size

In general, a node's size should *not* be defined *explicitly*. Doing so often creates a design that looks pleasing when it first loads, but deteriorates when the app is resized or the con-

1. The corresponding Swing GUI chapter is now online Chapter 35 and can be covered after online Chapter 26, which requires as prerequisites the only Chapters 1 through 11.

tent updates. In addition to the width and height properties, most JavaFX nodes have the properties **prefWidth**, **prefHeight**, **minWidth**, **minHeight**, **maxWidth** and **maxHeight** that specify a node's *range* of acceptable sizes as it's laid out within its parent node:

- The minimum size properties specify a node's smallest allowed size in points.
- The maximum size properties specify a node's largest allowed size in points.
- The preferred size properties specify a node's preferred width and height that should be used by the layout in most cases.

Node Position and Layout Panes

A node's position should be defined *relative* to its parent node and the other nodes in its parent. JavaFX **layout panes** are container nodes that arrange their child nodes in a scene graph relative to one another, based on their sizes and positions. Child nodes are controls, other layout panes, shapes and more.

Most JavaFX layout panes use *relative positioning*—if a layout-pane node is resized, it adjusts its children's sizes and positions accordingly, based on their preferred, minimum and maximum sizes. Figure 13.1 describes each of the JavaFX layout panes, including those presented in Chapter 12. In this chapter, we'll use Pane, BorderPane, GridPane and VBox from the `javafx.scene.layout` package.

Layout	Description
AnchorPane	Enables you to set the position of child nodes relative to the pane's edges. Resizing the pane does not alter the layout of the nodes.
BorderPane	Includes five areas—top, bottom, left, center and right—where you can place nodes. The top and bottom regions fill the BorderPane's width and are vertically sized to their children's preferred heights. The left and right regions fill the BorderPane's height and are horizontally sized to their children's preferred widths. The center area occupies all of the BorderPane's remaining space. You might use the different areas for tool bars, navigation, a main content area, etc.
FlowPane	Lays out nodes consecutively—either horizontally or vertically. When the boundary for the pane is reached, the nodes wrap to a new line in a horizontal FlowPane or a new column in a vertical FlowPane.
GridPane	Creates a flexible grid for laying out nodes in rows and columns.
Pane	The base class for layout panes. This can be used to position nodes at fixed locations—known as absolute positioning.
StackPane	Places nodes in a stack. Each new node is stacked atop the previous node. You might use this to place text on top of images, for example.
TilePane	A horizontal or vertical grid of equally sized tiles. Nodes that are tiled horizontally wrap at the TilePane's width. Nodes that are tiled vertically wrap at the TilePane's height.
HBox	Arranges nodes horizontally in one row.
VBox	Arranges nodes vertically in one column.

Fig. 13.1 | JavaFX layout panes.

13.3 Painter App: RadioButtons, Mouse Events and Shapes

In this section, you'll create a simple **Painter** app (Fig. 13.2) that allows you to drag the mouse to draw. First, we'll overview the technologies you'll use, then we'll discuss creating the app's project and building its GUI. Finally, we'll present the source code for its **Painter** and **PainterController** classes.

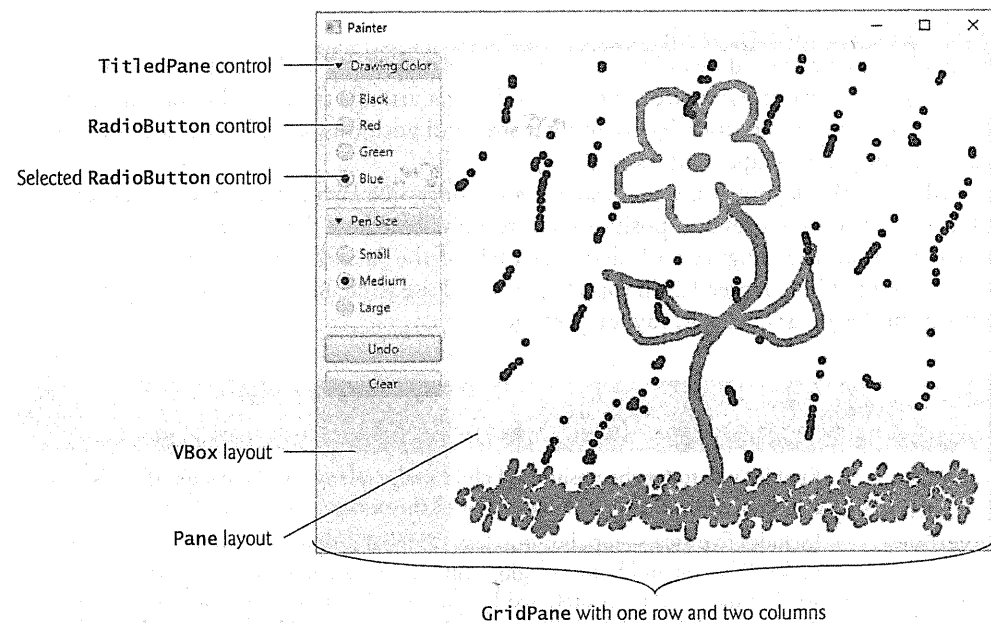


Fig. 13.2 | Painter app.

13.3.1 Technologies Overview

This section introduces the JavaFX features you'll use in the **Painter** app.

RadioButtons and ToggleGroups

RadioButtons function as *mutually exclusive* options. You add multiple **RadioButtons** to a **ToggleGroup** to ensure that only one **RadioButton** in a given group is selected at a time. For this app, you'll use JavaFX Scene Builder's capability for specifying each **RadioButton**'s **ToggleGroup** in FXML; however, you can also create a **ToggleGroup** in Java, then use a **RadioButton**'s **setToggleGroup** method to specify its **ToggleGroup**.

BorderPane Layout Container

A **BorderPane** layout container arranges controls into one or more of the five regions shown in Fig. 13.3. The top and bottom areas have the same width as the **BorderPane**. The left, center and right areas fill the vertical space between the top and bottom areas.

Each area may contain only one control or one layout container that, in turn, may contain other controls.



Look-and-Feel Observation 13.1

*All the areas in a **BorderPane** are optional: If the top or bottom area is empty, the left, center and right areas expand vertically to fill that area. If the left or right area is empty, the center expands horizontally to fill that area.*

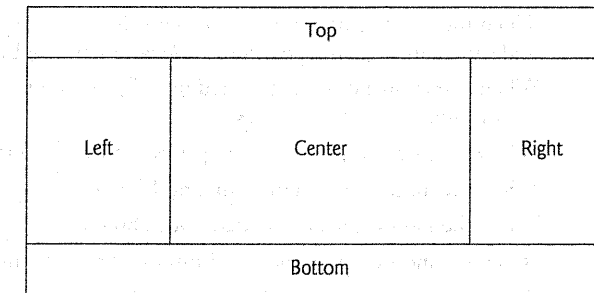


Fig. 13.3 | **BorderPane**'s five areas.

TitledPane Layout Container

A **TitledPane** layout container displays a title at its top and is a collapsible panel containing a layout node, which in turn contains other nodes. You'll use **TitledPanes** to organize the app's **RadioButtons** and to help the user understand the purpose of each **RadioButton** group.

JavaFX Shapes

The **javafx.scene.shape** package contains various classes for creating 2D and 3D shape nodes that can be displayed in a scene graph. In this app, you'll programmatically create **Circle** objects as the user drags the mouse, then attach them to the app's drawing area so that they're displayed in the scene graph.

Pane Layout Container

Each **Circle** you programmatically create is attached to an **Pane** layout (the drawing area) at a specified *x-y* coordinate measured from the **Pane**'s upper-left corner.

Mouse Event Handling

When you drag the mouse, the app's controller responds by displaying a **Circle** (in the currently selected color and pen size) at the current mouse position in the **Pane**. JavaFX nodes support various mouse events, which are summarized in Fig. 13.4. For this app, you'll configure an **onMouseDragged** event handler for the **Pane**. JavaFX also supports other types of input events. For example, for touchscreen devices there are various touch-oriented events and for keyboards there are various key events. For a complete list of JavaFX node events, see the **Node** class's properties that begin with the word "on" at:

<http://docs.oracle.com/javase/8/javafx/api/javafx/scene/Node.html>

Mouse events	When the event occurs for a given node
<code>onMouseClicked</code>	When the user clicks a mouse button—that is, presses and releases a mouse button without moving the mouse—with the mouse cursor within that node.
<code>onMouseDragEntered</code>	When the mouse cursor enters a node's bounds during a mouse drag—that is, the user is moving the mouse with a mouse button pressed.
<code>onMouseDragExited</code>	When the mouse cursor exits the node's bounds during a mouse drag.
<code>onMouseDragged</code>	When the user begins a mouse drag with the mouse cursor within that node and continues moving the mouse with a mouse button pressed.
<code>onMouseDragOver</code>	When a drag operation that started in a <i>different</i> node continues with the mouse cursor over the given node.
<code>onMouseDragReleased</code>	When the user completes a drag operation that began in that node.
<code>onMouseEntered</code>	When the mouse cursor enters that node's bounds.
<code>onMouseExited</code>	When the mouse cursor exits that node's bounds.
<code>onMouseMoved</code>	When the mouse cursor moves within that node's bounds.
<code>onMousePressed</code>	When user presses a mouse button with the mouse cursor within that node's bounds.
<code>onMouseReleased</code>	When user releases a mouse button with the mouse cursor within that node's bounds.

Fig. 13.4 | Mouse events.

Setting a Control's User Data

Each JavaFX control has a `setUserData` method that receives an `Object`. You can use this to store any object you'd like to associate with that control. With each drawing-color `RadioButton`, we store the specific `Color` that the `RadioButton` represents. With each pen size `RadioButton`, we store an enum constant for the corresponding pen size. We then use these objects when handling the `RadioButton` events.

13.3.2 Creating the Painter .fxml File

Create a folder on your system for this example's files, then open Scene Builder and save the new FXML file as `Painter.fxml`. If you already have an FXML file open, you also can choose **File > New** to create a new FXML file, then save it.

13.3.3 Building the GUI

In this section, we'll discuss the `Painter` app's GUI. Rather than providing the exact steps as we did in Chapter 12, we'll provide general instructions for building the GUI and focus on specific details for new concepts.



Software Engineering Observation 13.1

As you build a GUI, it's often easier to manipulate layouts and controls via Scene Builder's Hierarchy window than directly in the stage design area.

fx:id Property Values for This App's Controls

Figure 13.5 shows the `fx:id` properties of the `Painter` app's programmatically manipulated controls. As you build the GUI, you should set the corresponding `fx:id` properties in the FXML document, as we discussed in Chapter 12.

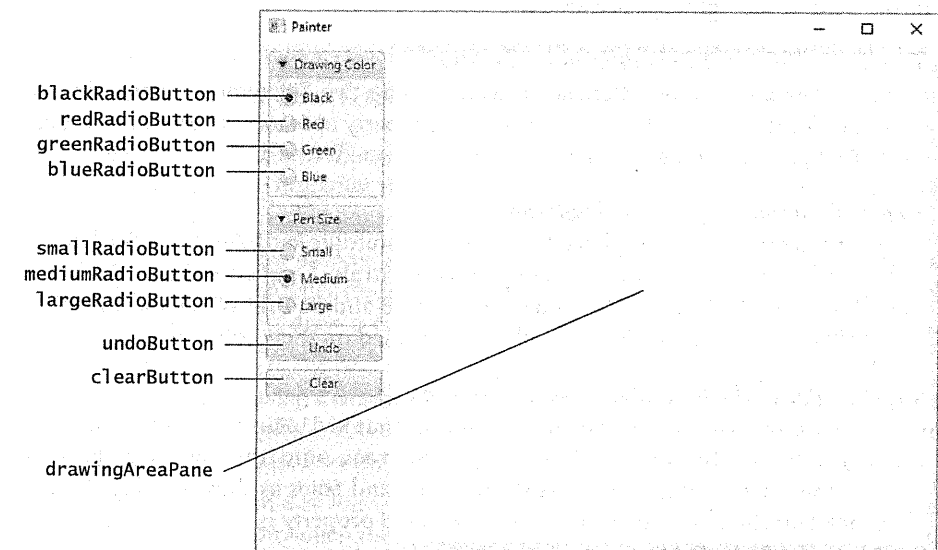


Fig. 13.5 | `Painter` GUI labeled with `fx:ids` for the programmatically manipulated controls.

Step 1: Adding a `BorderPane` as the Root Layout Node

Drag a `BorderPane` from the Scene Builder Library window's **Containers** section onto the content panel.

Step 2: Configuring the `BorderPane`

We set the `GridPane`'s **Pref Width** and **Pref Height** properties to 640 and 480 respectively. Recall that the stage's size is determined based on the size of the root node in the FXML document. Set the `BorderPane`'s **Padding** property to 8 to inset it from the stage's edges.

Step 3: Adding the `VBox` and `Pane`

Drag a `VBox` into the `BorderPane`'s left area and an `Pane` into the center area. As you drag over the `BorderPane`, Scene Builder shows the layout's five areas and highlights the area in which the item you're dragging will be placed when you release the mouse. Set the `Pane`'s `fx:id` to `drawingAreaPane` as specified in Fig. 13.5.

For the `VBox`, set its **Spacing** property (in the **Inspector**'s **Layout** section) to 8 to add some vertical spacing between the controls that will be added to this container. Set its right **Margin** property to 8 to add some horizontal spacing between the `VBox` and the `Pane` be added to this container. Also reset its **Pref Width** and **Pref Height** properties to their default values (`USE_COMPUTED_SIZE`) and set its **Max Height** property to `MAX_VALUE`. This will enable the `VBox` to be as wide as it needs to be to accommodate its child nodes and occupy the full column height.

Reset the Pane's **Pref Width** and **Pref Height** to their default `USE_COMPUTED_SIZE` values, and set its **Max Width** and **Max Height** to `MAX_VALUE` so that it occupies the full width and height of the `BorderPane`'s center area. In the **JavaFX CSS** category of the **Inspector** window's **Properties** section, click the field below **Style** (which is initially empty) and select `-fx-background-color` to indicate that you'd like to specify the Pane's background color. In the field to the right, specify `white`.

Step 4: Adding the **TitledPanes** to the **VBox**

From the **Library** window's **Containers** section, drag two **TitledPane** (empty) objects onto the **VBox**. For the first **TitledPane**, set its **Text** property to **Drawing Color**. For the second, set its **Text** property to **Pen Size**.

Step 5: Customizing the **TitledPanes**

Each **TitledPane** in the completed GUI contains multiple **RadioButtons**. We'll use a **VBox** within each **TitledPane** to help arrange those controls. Drag a **VBox** onto each **TitledPane**. For each **VBox**, set its **Spacing** property to 8 and its **Pref Width** and **Pref Height** to `USE_COMPUTED_SIZE` so the **VBoxes** will be sized based on their contents.

Step 6: Adding the **RadioButtons** to the **VBox**

From the **Library** window's **Controls** section, drag four **RadioButtons** onto the **VBox** for the **Drawing Color** **TitledPane**, and three **RadioButtons** onto the **VBox** for the **Pen Size** **TitledPane**, then configure their **Text** properties and **fx:ids** as shown in Fig. 13.5. Select the **blackRadioButton** and ensure that its **Selected** property is checked, then do the same for the **mediumRadioButton**.

Step 7: Specifying the **ToggleGroups** for the **RadioButtons**

Select all four **RadioButtons** in the first **TitledPane**'s **VBox**, then set the **Toggle Group** property to `colorToggleGroup`. When the **FXML** file is loaded, a **ToggleGroup** object by that name will be created and these four **RadioButtons** will be associated with it to ensure that only one is selected at a time. Repeat this step for the three **RadioButtons** in the second **TitledPane**'s **VBox**, but set the **Toggle Group** property to `sizeToggleGroup`.

Step 8: Changing the **TitledPanes**' Preferred Width and Height

For each **TitledPane**, set its **Pref Width** and **Pref Height** to `USE_COMPUTED_SIZE` so the **TitledPanes** will be sized based on their contents.

Step 9: Adding the **Buttons**

Add two **Buttons** below the **TitledPanes**, then configure their **Text** properties and **fx:ids** as shown in Fig. 13.5. Set each **Button**'s **Max Width** property to `MAX_VALUE` so that they fill the **VBox**'s width.

Step 10: Setting the Width the **VBox**

We'd like the **VBox** to be only as wide as it needs to be to display the controls in that column. To specify this, select the **VBox** in the **Document** window's **Hierarchy** section. Set the column's **Min Width** and **Pref Width** to `USE_COMPUTED_SIZE`, then set the **Max Width** to `USE_PREF_SIZE` (which indicates that the maximum width should be the preferred width). Also, reset the **Max Height** to its default `USE_COMPUTED_SIZE` value. The GUI is now complete and should appear as shown in Fig. 13.5.

Step 11: Specifying the **Controller Class's Name**

As we mentioned in Section 12.5.2, in a **JavaFX FXML** app, the app's controller class typically defines instance variables for interacting with controls programmatically, as well as event-handling methods. To ensure that an object of the controller class is created when the app loads the **FXML** file at runtime, you must specify the controller class's name in the **FXML** file:

1. Expand **Scene Builder**'s **Controller** window (located below the **Hierarchy** window).
2. In the **Controller Class** field, type `PainterController`.

Step 12: Specifying the **Event-Handler Method Names**

Next, you'll specify in the **Inspector** window's **Code** section the names of the methods that will be called to handle specific control's events:

- For the **drawingAreaPane**, specify `drawingAreaMouseDragged` as the **On Mouse Dragged** event handler (located under the **Mouse** heading in the **Code** section). This method will draw a circle in the specified color and size for each mouse-dragged event.
- For the four **Drawing Color** **RadioButtons**, specify `colorRadioButtonSelected` as each **RadioButton**'s **On Action** event handler. This method will set the current drawing color, based on the user's selection.
- For the three **Pen Size** **RadioButtons**, specify `sizeRadioButtonSelected` as each **RadioButton**'s **On Action** event handler. This method will set the current pen size, based on the user's selection.
- For the **Undo** **Button**, specify `undoButtonPressed` as the **On Action** event handler. This method will remove the last circle the user drew on the screen.
- For the **Clear** **Button**, specify `clearButtonPressed` as the **On Action** event handler. This method will clear the entire drawing.

Step 13: Generating a **Sample Controller Class**

As you saw in Section 12.5, **Scene Builder** generates the initial controller-class skeleton for you when you select **View > Show Sample Controller Skeleton**. You can copy this code into a `PainterController.java` file and store the file in the same folder as `Painter.fxml`. We show the completed `PainterController` class in Section 13.3.5.

13.3.4 **Painter** Subclass of **Application**

Figure 13.6 shows class `Painter` subclass of `Application` that launches the app, which performs the same tasks to start the `Painter` app as described for the `Tip Calculator` app in Section 12.5.4.

```
1 // Fig. 13.5: Painter.java
2 // Main application class that loads and displays the Painter's GUI.
3 import javafx.application.Application;
4 import javafx.fxml.FXMLLoader;
```

Fig. 13.6 | Main application class that loads and displays the **Painter**'s GUI.

```

5 import javafx.scene.Parent;
6 import javafx.scene.Scene;
7 import javafx.stage.Stage;
8
9 public class Painter extends Application {
10     @Override
11     public void start(Stage stage) throws Exception {
12         Parent root =
13             FXMLLoader.load(getClass().getResource("Painter.fxml"));
14
15         Scene scene = new Scene(root);
16         stage.setTitle("Painter"); // displayed in window's title bar
17         stage.setScene(scene);
18         stage.show();
19     }
20
21     public static void main(String[] args) {
22         launch(args);
23     }
24 }

```

Fig. 13.6 | Main application class that loads and displays the Painter's GUI.

13.3.5 PainterController Class

Figure 13.7 shows the final version of class `PainterController` with this app's new features highlighted. Recall from Chapter 12 that the controller class defines instance variables for interacting with controls programmatically, as well as event-handling methods. The controller class may also declare additional instance variables, static variables and methods that support the app's operation.

```

1 // Fig. 13.6: PainterController.java
2 // Controller for the Painter app
3 import javafx.event.ActionEvent;
4 import javafx.fxml.FXML;
5 import javafx.scene.control.RadioButton;
6 import javafx.scene.control.ToggleGroup;
7 import javafx.scene.input.MouseEvent;
8 import javafx.scene.layout.Pane;
9 import javafx.scene.paint.Color;
10 import javafx.scene.paint.Paint;
11 import javafx.scene.shape.Circle;
12
13 public class PainterController {
14     // enum representing pen sizes
15     private enum PenSize {
16         SMALL(2),
17         MEDIUM(4),
18         LARGE(6);

```

Fig. 13.7 | Controller for the Painter app. (Part 1 of 3.)

```

19
20     private final int radius;
21
22     PenSize(int radius) {this.radius = radius;} // constructor
23
24     public int getRadius() {return radius;}
25 };
26
27 // instance variables that refer to GUI components
28 @FXML private RadioButton blackRadioButton;
29 @FXML private RadioButton redRadioButton;
30 @FXML private RadioButton greenRadioButton;
31 @FXML private RadioButton blueRadioButton;
32 @FXML private RadioButton smallRadioButton;
33 @FXML private RadioButton mediumRadioButton;
34 @FXML private RadioButton largeRadioButton;
35 @FXML private Pane drawingAreaPane;
36 @FXML private ToggleGroup colorToggleGroup;
37 @FXML private ToggleGroup sizeToggleGroup;
38
39 // instance variables for managing Painter state
40 private PenSize radius = PenSize.MEDIUM; // radius of circle
41 private Paint brushColor = Color.BLACK; // drawing color
42
43 // set user data for the RadioButtons
44 public void initialize() {
45     // user data on a control can be any Object
46     blackRadioButton.setUserData(Color.BLACK);
47     redRadioButton.setUserData(Color.RED);
48     greenRadioButton.setUserData(Color.GREEN);
49     blueRadioButton.setUserData(Color.BLUE);
50     smallRadioButton.setUserData(PenSize.SMALL);
51     mediumRadioButton.setUserData(PenSize.MEDIUM);
52     largeRadioButton.setUserData(PenSize.LARGE);
53 }
54
55 // handles drawingArea's onMouseDragged MouseEvent
56 @FXML
57 private void drawingAreaMouseDragged(MouseEvent e) {
58     Circle newCircle = new Circle(e.getX(), e.getY(),
59         radius.getRadius(), brushColor);
60     drawingAreaPane.getChildren().add(newCircle);
61 }
62
63 // handles color RadioButton's ActionEvents
64 @FXML
65 private void colorRadioButtonSelected(ActionEvent e) {
66     // user data for each color RadioButton is the corresponding Color
67     brushColor =
68         (Color) colorToggleGroup.getSelectedToggle().getUserData();
69 }
70

```

Fig. 13.7 | Controller for the Painter app. (Part 2 of 3.)

```

71 // handles size RadioButton's ActionEvents
72 @FXML
73 private void sizeRadioButtonSelected(ActionEvent e) {
74     // user data for each size RadioButton is the corresponding PenSize
75     radius =
76     (PenSize) sizeToggleGroup.getSelectedToggle().getUserData();
77 }
78
79 // handles Undo Button's ActionEvents
80 @FXML
81 private void undoButtonPressed(ActionEvent event) {
82     int count = drawingAreaPane.getChildren().size();
83
84     // if there are any shapes remove the last one added
85     if (count > 0) {
86         drawingAreaPane.getChildren().remove(count - 1);
87     }
88 }
89
90 // handles Clear Button's ActionEvents
91 @FXML
92 private void clearButtonPressed(ActionEvent event) {
93     drawingAreaPane.getChildren().clear(); // clear the canvas
94 }
95 }

```

Fig. 13.7 | Controller for the Painter app. (Part 3 of 3.)

PenSize enum

Lines 15–25 define the nested enum type `PenSize`, which specifies three pen sizes—`SMALL`, `MEDIUM` and `LARGE`. Each has a corresponding radius that will be used when creating a `Circle` object to display in response to a mouse-drag event.

Java allows you to declare classes, interfaces and enums as **nested types** inside other classes. Except for the anonymous inner class introduced in Section 12.5.5, all the classes, interfaces and enums we've discussed were **top level**—that is, they *were* not declared *inside* another type. The enum type `PenSize` is declared here as a **private nested type** because it's used only by class `PainterController`. We'll say more about nested types later in the book.

Instance Variables

Lines 28–37 declare the `@FXML` instance variables that the controller uses to programmatically interact with the GUI. Recall that the names of these variables must match the corresponding `fx:id` values that you specified in `Painter.fxml`; otherwise, the `FXMLLoader` will not be able to connect the GUI components to the instance variables. Two of the `@FXML` instance variables are `ToggleGroups`—in the `RadioButton` event handlers, we'll use these to determine which `RadioButton` was selected. Lines 40–41 define two additional instance variables that store the current drawing `Color` and the current `PenSize`, respectively.

Method initialize

Recall that when the `FXMLLoader` creates a controller-class object, `FXMLLoader` determines whether the class contains an `initialize` method with no parameters and, if so, calls that

method to initialize the controller. Lines 44–53 define method `initialize` to specify each `RadioButton`'s corresponding user data object—either a `Color` or a `PenSize`. You'll use these objects in the `RadioButtons`' event handlers.

drawingAreaMouseDragged Event Handler

Lines 56–61 define `drawingAreaMouseDragged`, which responds to drag events in the `drawingAreaPane`. Each mouse event handler you define must have one `MouseEvent` parameter (package `javafx.scene.input`). When the event occurs, this parameter contains information about the event, such as its location, whether any mouse buttons were pressed, which node the user interacted with and more. You specified `drawingAreaMouseDragged` in Scene Builder as the `drawingAreaPane`'s **On Mouse Dragged** event handler.

Lines 58–59 create a new `Circle` object using the constructor that takes as arguments the center point's *x*-coordinate, the center point's *y*-coordinate, the `Circle`'s radius and the `Circle`'s `Color`.

Next, line 60 attaches the new `Circle` to the `drawingAreaPane`. Each layout pane has a `getChildren` method that returns an `ObservableList<Node>` collection containing the layout's child nodes. An `ObservableList` provides methods for adding and removing elements. You'll learn more about `ObservableList` later in this chapter. Line 60 uses the `ObservableList`'s `add` method to add a new `Node` to the `drawingAreaPane`—all JavaFX shapes inherit indirectly from class `Node` in the `javafx.scene` package.

colorRadioButtonSelected Event Handler

Lines 64–69 define `colorRadioButtonSelected`, which responds to the `ActionEvents` of the **Drawing Color** `RadioButtons`—these occur each time a new color `RadioButton` is selected. You specified this event handler in Scene Builder as the **On Action** event handler for all four **Drawing Color** `RadioButtons`.

Lines 67–68 set the current drawing `Color`. `ColorToggleGroup` method `getSelectedToggle` returns the `Toggle` that's currently selected. Class `RadioButton` is one of several controls (others are `RadioButtonMenuItem` and `ToggleButton`) that implement interface `Toggle`. We then use the `Toggle`'s `getUserData` method to get the user data `Object` that was associated with the corresponding `RadioButton` in method `initialize`. For the color `RadioButtons`, this `Object` is always a `Color`, so we cast the `Object` to a `Color` and assign it to `brushColor`.

sizeRadioButtonSelected Event Handler

Lines 72–77 define `sizeRadioButtonSelected`, which responds to the pen size `RadioButtons`' `ActionEvents`. You specified this event handler as the **On Action** event handler for all three **Pen Size** `RadioButtons`. Lines 75–76 set the current `PenSize`, using the same approach as setting the current color in method `colorRadioButtonSelected`.

undoButtonPressed Event Handler

Lines 80–88 define `undoButtonPressed`, which responds to an `ActionEvent` from the `undoButton` by removing the last `Circle` displayed. You specified this event handler in Scene Builder as the `undoButton`'s **On Action** event handler.

To undo the last `Circle`, we remove the last child from the `drawingAreaPane`'s collection of child nodes. First, line 82 gets the number of elements in that collection. Then, if that's greater than 0, line 86 removes the node at the last index in the collection.

clearButtonPressed Event Handler

Lines 91–94 define `clearButtonPressed`, which responds to the `ActionEvent` from the `clearButton` by clearing `drawingAreaPane`'s collection of child nodes. You specified this event handler in Scene Builder as the `clearButton`'s On Action event handler. Line 93 clears the collection of child nodes to erase the entire drawing.

13.4 Color Chooser App: Property Bindings and Property Listeners

In this section, we present a **Color Chooser** app (Fig. 13.8) that demonstrates property bindings and property listeners.

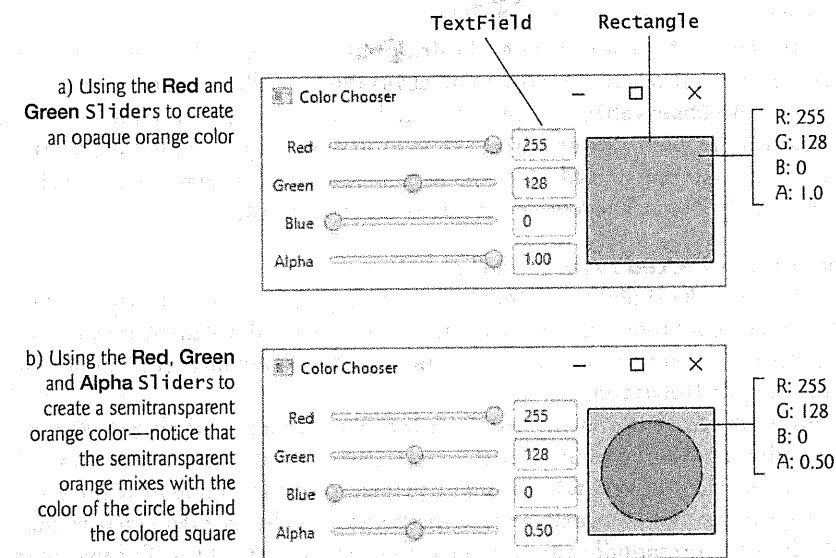


Fig. 13.8 | Color Chooser app with opaque and semitransparent orange colors.

13.4.1 Technologies Overview

In this section, we introduce the technologies you'll use to build the **Color Chooser**.

RGBA Colors

The app uses the **RGBA color system** to display a rectangle of color based on the values of four `Slider`s. In **RGBA**, every color is represented by its red, green and blue color values, each ranging from 0 to 255, where 0 denotes no color and 255 full color. For example, a color with a red value of 0 would contain no red component. The alpha value (A)—which ranges from 0.0 to 1.0—represents a color's *opacity*, with 0.0 being completely *transparent* and 1.0 completely *opaque*. The two colors in Fig. 13.8's sample outputs have the same RGB values, but the color displayed in Fig. 13.8(b) is *semitransparent*. You'll use a `Color` object that's created with **RGBA** values to fill a `Rectangle` that displays the `Color`.

Properties of a Class

JavaFX makes extensive use of properties. A **property** is defined by creating *set* and *get* methods with specific naming conventions. In general, the pair of methods that define a read/write property have the form:

```
public final void setPropertyName(Type propertyName)
public final Type getPropertyName()
```

Typically, such methods manipulate a corresponding *private* instance variable that has the same name as the property, but this is not required. For example, methods `setHour` and `getHour` together represent a property named `hour` and typically would manipulate a *private* `hour` instance variable. If the property represents a boolean value, its *get* method name typically begins with "is" rather than "get"—for example, `ArrayList` method `isEmpty`.



Software Engineering Observation 13.2

Methods that define properties should be declared `final` to prevent subclasses from overriding the methods, which could lead to unexpected results in client code.

Property Bindings

JavaFX properties are implemented in a manner that makes them *observable*—when a property's value changes, other objects can respond accordingly. This is similar to event handling. One way to respond to a property change is via a **property binding**, which enables a property of one object to be updated when a property of another object changes. For example, you'll use property bindings to enable a `TextField` to display the corresponding `Slider`'s current value when the user moves that `Slider`'s thumb. Property bindings are not limited to JavaFX controls. Package `javafx.beans.property` contains many classes that you can use to define bindable properties in your own classes.

Property Listeners

Property listeners are similar to property bindings. A **property listener** is an event handler that's invoked when a property's value changes. In the event handler, you can respond to the property change in a manner appropriate for your app. In this app, when a `Slider`'s value changes, a property listener will store the value in a corresponding instance variable, create a new `Color` based on the values of all four `Slider`s and set that `Color` as the fill color of a `Rectangle` object that displays the current color. For more information on properties, property bindings and property listeners, visit:

<http://docs.oracle.com/javase/8/javafx/properties-binding-tutorial/binding.htm>

13.4.2 Building the GUI

In this section, we'll discuss the **Color Chooser** app's GUI. Rather than providing the exact steps as we did in Chapter 12, we'll provide general instructions for building the GUI and focus on specific details for new concepts. As you build the GUI, recall that it's often easier to manipulate layouts and controls via the Scene Builder **Document** window's **Hierarchy** section than directly in the stage design area. Before proceeding, open Scene Builder and create an `FXML` file named `ColorChooser.fxml`.

fx:id Property Values for This App's Controls

Figure 13.9 shows the fx:id properties of the Color Chooser app's programmatically manipulated controls. As you build the GUI, you should set the corresponding fx:id properties in the FXML document, as you learned in Chapter 12.

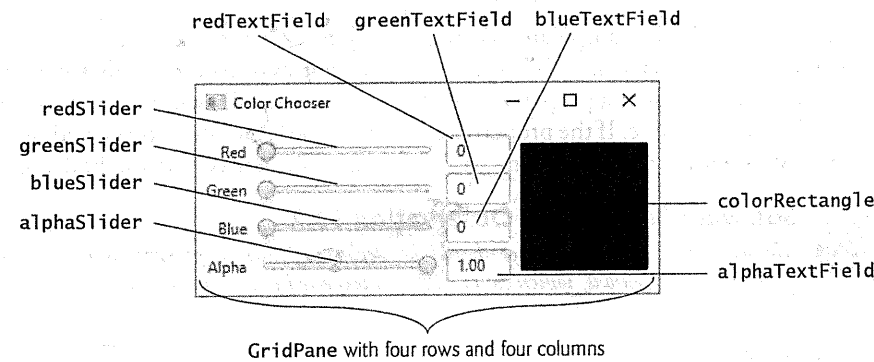


Fig. 13.9 | Color Chooser app's programmatically manipulated controls labeled with their fx:ids.

Step 1: Adding a GridPane

Drag a GridPane from the Library window's Containers section onto Scene Builder's content panel.

Step 2: Configuring the GridPane

This app's GridPane requires four rows and four columns. Use the techniques you've learned previously to add two columns and one row to the GridPane. Set the GridPane's Hgap and Padding properties to 8 to inset the GridPane from the stage's edges and to provide space between its columns.

Step 3: Adding the Controls

Using Fig. 13.9 as a guide, add the Labels, Sliders, TextFields, a Circle and a Rectangle to the GridPane—Circle and Rectangle are located in the Scene Builder Library's Shapes section. When adding the Circle and Rectangle, place both into the rightmost column's first row. Be sure to add the Circle *before* the Rectangle so that it will be located *behind* the rectangle in the layout. Set the text of the Labels and TextFields as shown and set all the appropriate fx:id properties as you add each control.

Step 4: Configuring the Sliders

For the red, green and blue Sliders, set the Max properties to 255 (the maximum amount of a given color in the RGBA color scheme). For the alpha Slider, set its Max property to 1.0 (the maximum opacity in the RGBA color scheme).

Step 5: Configuring the TextFields

Set all of the TextField's Pref Width properties to 50.

Step 6: Configuring the Rectangle

Set the Rectangle's Width and Height properties to 100, then set its Row Span property to Remainder so that it spans all four rows.

Step 7: Configuring the Circle

Set the Circle's Radius property to 40, then set its Row Span property to Remainder so that it spans all four rows.

Step 8: Configuring the Rows

Set all four columns' Pref Height properties to USE_COMPUTED_SIZE so that the rows are only as tall as their content.

Step 9: Configuring the Columns

Set all four columns' Pref Width properties to USE_COMPUTED_SIZE so that the columns are only as wide as their content. For the leftmost column, set the Halignment property to RIGHT. For the rightmost column, set the Halignment property to CENTER.

Step 10: Configuring the GridPane

Set the GridPane's Pref Width and Pref Height properties to USE_COMPUTED_SIZE so that it sizes itself, based on its contents. Your GUI should now appear as shown in Fig. 13.9.

Step 11: Specifying the Controller Class's Name

To ensure that an object of the controller class is created when the app loads the FXML file at runtime, specify ColorChooserController as the controller class's name in the FXML file as you've done previously.

Step 12: Generating a Sample Controller Class

Select View > Show Sample Controller Skeleton, then copy this code into a ColorChooserController.java file and store the file in the same folder as ColorChooser.fxml. We show the completed ColorChooserController class in Section 13.4.4.

13.4.3 ColorChooser Subclass of Application

Figure 13.6 shows the ColorChooser subclass of Application that launches the app. This class loads the FXML and displays the app as in the prior JavaFX examples.

```

1 // Fig. 13.8: ColorChooser.java
2 // Main application class that loads and displays the ColorChooser's GUI.
3 import javafx.application.Application;
4 import javafx.fxml.FXMLLoader;
5 import javafx.scene.Parent;
6 import javafx.scene.Scene;
7 import javafx.stage.Stage;
8
9 public class ColorChooser extends Application {
10     @Override
11     public void start(Stage stage) throws Exception {
12         Parent root =
13             FXMLLoader.load(getClass().getResource("ColorChooser.fxml"));

```

Fig. 13.10 | Application class that loads and displays the Color Chooser's GUI. (Part 1 of 2.)

```

14
15     Scene scene = new Scene(root);
16     stage.setTitle("Color Chooser");
17     stage.setScene(scene);
18     stage.show();
19 }
20
21 public static void main(String[] args) {
22     launch(args);
23 }
24 }

```

Fig. 13.10 | Application class that loads and displays the Color Chooser's GUI. (Part 2 of 2.)

13.4.4 ColorChooserController Class

Figure 13.11 shows the final version of class ColorChooserController with this app's new features highlighted.

```

1 // Fig. 13.9: ColorChooserController.java
2 // Controller for the ColorChooser app
3 import javafx.beans.value.ChangeListener;
4 import javafx.beans.value.ObservableValue;
5 import javafx.fxml.FXML;
6 import javafx.scene.control.Slider;
7 import javafx.scene.control.TextField;
8 import javafx.scene.paint.Color;
9 import javafx.scene.shape.Rectangle;
10
11 public class ColorChooserController {
12     // instance variables for interacting with GUI components
13     @FXML private Slider redSlider;
14     @FXML private Slider greenSlider;
15     @FXML private Slider blueSlider;
16     @FXML private Slider alphaSlider;
17     @FXML private TextField redTextField;
18     @FXML private TextField greenTextField;
19     @FXML private TextField blueTextField;
20     @FXML private TextField alphaTextField;
21     @FXML private Rectangle colorRectangle;
22
23     // instance variables for managing
24     private int red = 0;
25     private int green = 0;
26     private int blue = 0;
27     private double alpha = 1.0;
28
29     public void initialize() {
30         // bind TextField values to corresponding Slider values
31         redTextField.textProperty().bind(
32             redSlider.valueProperty().asString("%.0f"));

```

Fig. 13.11 | Controller for the ColorChooser app. (Part 1 of 2.)

```

33         greenTextField.textProperty().bind(
34             greenSlider.valueProperty().asString("%.0f"));
35         blueTextField.textProperty().bind(
36             blueSlider.valueProperty().asString("%.0f"));
37         alphaTextField.textProperty().bind(
38             alphaSlider.valueProperty().asString("%.2f"));
39
40         // listeners that set Rectangle's fill based on Slider changes
41         redSlider.valueProperty().addListener(
42             new ChangeListener<Number>() {
43                 @Override
44                 public void changed(ObservableValue<? extends Number> ov,
45                     Number oldValue, Number newValue) {
46                     red = newValue.intValue();
47                     colorRectangle.setFill(Color.rgb(red, green, blue, alpha));
48                 }
49             });
50
51         greenSlider.valueProperty().addListener(
52             new ChangeListener<Number>() {
53                 @Override
54                 public void changed(ObservableValue<? extends Number> ov,
55                     Number oldValue, Number newValue) {
56                     green = newValue.intValue();
57                     colorRectangle.setFill(Color.rgb(red, green, blue, alpha));
58                 }
59             });
60
61         blueSlider.valueProperty().addListener(
62             new ChangeListener<Number>() {
63                 @Override
64                 public void changed(ObservableValue<? extends Number> ov,
65                     Number oldValue, Number newValue) {
66                     blue = newValue.intValue();
67                     colorRectangle.setFill(Color.rgb(red, green, blue, alpha));
68                 }
69             });
70
71         alphaSlider.valueProperty().addListener(
72             new ChangeListener<Number>() {
73                 @Override
74                 public void changed(ObservableValue<? extends Number> ov,
75                     Number oldValue, Number newValue) {
76                     alpha = newValue.doubleValue();
77                     colorRectangle.setFill(Color.rgb(red, green, blue, alpha));
78                 }
79             });
80     }
81 }
82 }

```

Fig. 13.11 | Controller for the ColorChooser app. (Part 2 of 2.)

Instance Variables

Lines 13–27 declare the controller's instance variables. Variables `red`, `green`, `blue` and `alpha` store the current values of the `redSlider`, `greenSlider`, `blueSlider` and `alphaSlider`, respectively. These values are used to update the `colorRectangle`'s fill color each time the user moves a `Slider`'s thumb.

Method initialize

Lines 29–81 define method `initialize`, which initializes the controller after the GUI is created. In this app, `initialize` configures the property bindings and property listeners.

Property-to-Property Bindings

Lines 31–38 set up property bindings between a `Slider`'s value and the corresponding `TextField`'s text so that changing a `Slider` updates the corresponding `TextField`. Consider lines 31–32, which bind the `redSlider`'s `valueProperty` to the `redTextField`'s `textProperty`:

```
redTextField.textProperty().bind(  
    redSlider.valueProperty().asString("%.0f"));
```

Each `TextField` has a text property that's returned by its `textProperty` method as a `StringProperty` (package `javafx.beans.property`). `StringProperty` method `bind` receives an `ObservableValue` as an argument. When the `ObservableValue` changes, the bound property updates accordingly. In this case the `ObservableValue` is the result of the expression `redSlider.valueProperty().asString("%.0f")`. `Slider`'s `valueProperty` method returns the `Slider`'s value property as a `DoubleProperty`—an observable double value. Because the `TextField`'s text property must be bound to a `String`, we call `DoubleProperty` method `asString`, which returns a `StringBinding` object (an `ObservableValue`) that produces a `String` representation of the `DoubleProperty`. This version of `asString` receives a format-control `String` specifying the `DoubleProperty`'s format.

Property Listeners

To perform an arbitrary task when a property's value changes, register a property listener. Lines 41–80 register property listeners for the `Sliders`' value properties. Consider lines 41–50, which register the `ChangeListener` that executes when the user moves the `redSlider`'s thumb. As we did in Section 12.5 for the **Tip Calculator**'s `Slider`, we use an anonymous inner class to define the listener. Each `ChangeListener` stores the `int` value of the `newValue` parameter in a corresponding instance variable, then calls the `colorRectangle`'s `setFill` method to change its color, using `Color` method `rgb` to create the new `Color` object.

13.5 Cover Viewer App: Data-Driven GUIs with JavaFX Collections

Often an app needs to edit and display data. JavaFX provides a comprehensive model for allowing GUIs to interact with data. In this section, you'll build the **Cover Viewer** app (Fig. 13.12), which binds a list of `Book` objects to a `ListView`. When the user selects an item in the `ListView`, the corresponding `Book`'s cover image is displayed in an `ImageView`.

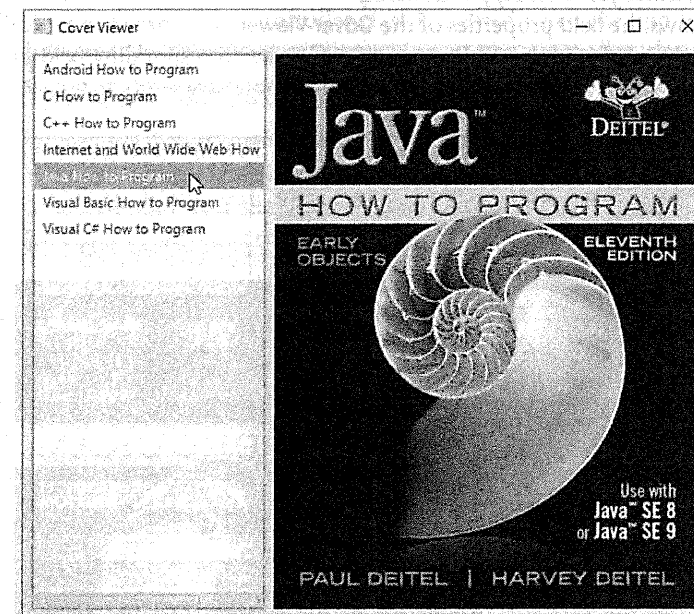


Fig. 13.12 | Cover Viewer with *Java How to Program* selected.

13.5.1 Technologies Overview

This app uses a `ListView` control to display a collection of book titles. Though you can individually add items to a `ListView`, in this app you'll bind an `ObservableList` object to the `ListView`. If you make changes to an `ObservableList`, its observer (the `ListView` in this app) will automatically be notified of those changes. Package `javafx.collections` defines `ObservableList` (similar to an `ArrayList`) and other observable collection interfaces. The package also contains class `FXCollections`, which provides static methods for creating and manipulating observable collections. You'll use a property listener to display the correct image when the user selects an item from the `ListView`—in this case, the property that changes is the selected item.

13.5.2 Adding Images to the App's Folder

From this chapter's examples folder, copy the `images` folder (which contains the `large` and `small` subfolders) into the folder where you'll save this app's FXML file, and the source-code files `CoverViewer.java` and `CoverViewerController.java`. Though you'll use only the `large` images in this example, you'll copy this app's folder to create the next example, which uses both sets of images.

13.5.3 Building the GUI

In this section, we'll discuss the **Cover Viewer** app's GUI. As you've done previously, create a new FXML file, then save it as `CoverViewer.fxml`.

fx:id Property Values for This App's Controls

Figure 13.13 shows the fx:id properties of the Cover Viewer app's programmatically manipulated controls. As you build the GUI, you should set the corresponding fx:id properties in the FXML document.

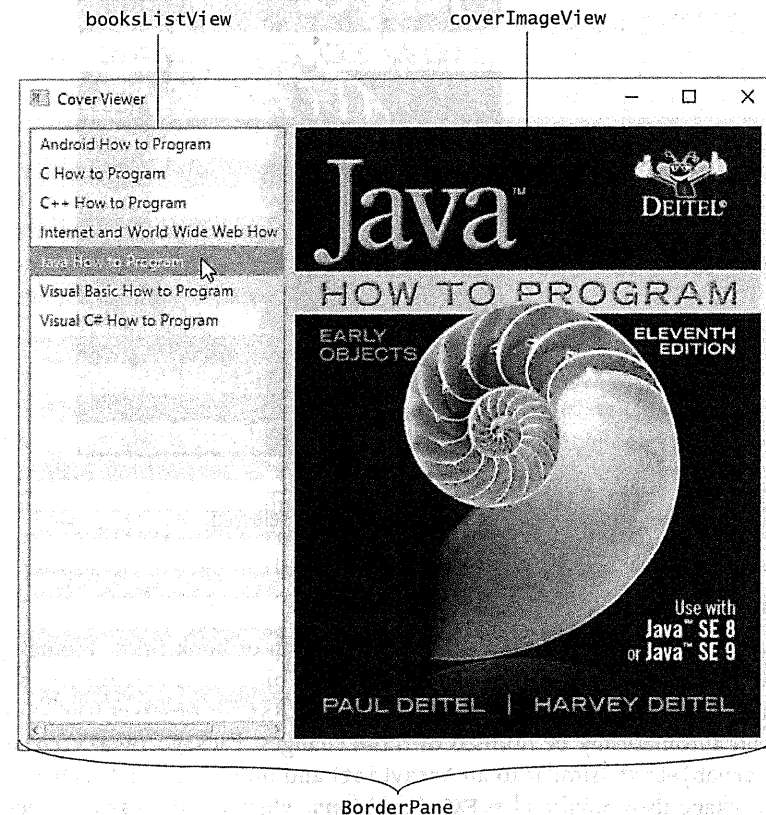


Fig. 13.13 | Cover Viewer app's programmatically manipulated controls labeled with their fx:ids.

Adding and Configuring the Controls

Using the techniques you learned previously, create a `BorderPane`. In the left area, place a `ListView` control, and in the center area, place an `ImageView` control.

For the `ListView`, set the following properties:

- **Margin**—8 (for the right margin) to separate the `ListView` from the `ImageView`
- **Pref Width**—200
- **Max Height**—`MAX_VALUE`
- **Min Width, Min Height, Pref Height and Max Width**—`USE_COMPUTED_SIZE`

For the `ImageView`, set the **Fit Width** and **Fit Height** properties to 370 and 480, respectively. To size the `BorderPane` based on its contents, set its **Pref Width** and **Pref Height** to `USE_COMPUTED_SIZE`. Also, set the **Padding** property to 8 to inset the `BorderPane` from the stage.

Specifying the Controller Class's Name

To ensure that an object of the controller class is created when the app loads the FXML file at runtime, specify `CoverViewerController` as the controller class's name in the FXML file as you've done previously.

Generating a Sample Controller Class

Select **View > Show Sample Controller Skeleton**, then copy this code into a `CoverViewerController.java` file and store the file in the same folder as `CoverViewer.fxml`. We show the completed `CoverViewerController` class in Section 13.5.5.

13.5.4 CoverViewer Subclass of Application

Figure 13.14 shows class `CoverViewer` subclass of `Application`.

```
1 // Fig. 13.13: CoverViewer.java
2 // Main application class that loads and displays the CoverViewer's GUI.
3 import javafx.application.Application;
4 import javafx.fxml.FXMLLoader;
5 import javafx.scene.Parent;
6 import javafx.scene.Scene;
7 import javafx.stage.Stage;
8
9 public class CoverViewer extends Application {
10     @Override
11     public void start(Stage stage) throws Exception {
12         Parent root =
13             FXMLLoader.load(getClass().getResource("CoverViewer.fxml"));
14
15         Scene scene = new Scene(root);
16         stage.setTitle("Cover Viewer");
17         stage.setScene(scene);
18         stage.show();
19     }
20
21     public static void main(String[] args) {
22         launch(args);
23     }
24 }
```

Fig. 13.14 | Main application class that loads and displays the Cover Viewer's GUI.

13.5.5 CoverViewerController Class

Figure 13.15 shows the final version of class `CoverViewerController` with the app's new features highlighted.

```
1 // Fig. 13.14: CoverViewerController.java
2 // Controller for Cover Viewer application
3 import javafx.beans.value.ChangeListener;
```

Fig. 13.15 | Controller for Cover Viewer application. (Part 1 of 2.)

```

4 import javafx.beans.value.ObservableValue;
5 import javafx.collections.FXCollections;
6 import javafx.collections.ObservableList;
7 import javafx.fxml.FXML;
8 import javafx.scene.control.ListView;
9 import javafx.scene.image.Image;
10 import javafx.scene.image.ImageView;
11
12 public class CoverViewerController {
13     // instance variables for interacting with GUI
14     @FXML private ListView<Book> booksListView;
15     @FXML private ImageView coverImageView;
16
17     // stores the list of Book Objects
18     private final ObservableList<Book> books =
19         FXCollections.observableArrayList();
20
21     // initialize controller
22     public void initialize() {
23         // populate the ObservableList<Book>
24         books.add(new Book("Android How to Program",
25             "/images/small/androidhttp.jpg", "/images/large/androidhttp.jpg"));
26         books.add(new Book("C How to Program",
27             "/images/small/chtp.jpg", "/images/large/chtp.jpg"));
28         books.add(new Book("C++ How to Program",
29             "/images/small/cpphttp.jpg", "/images/large/cpphttp.jpg"));
30         books.add(new Book("Internet and World Wide Web How to Program",
31             "/images/small/iw3http.jpg", "/images/large/iw3http.jpg"));
32         books.add(new Book("Java How to Program",
33             "/images/small/jhttp.jpg", "/images/large/jhttp.jpg"));
34         books.add(new Book("Visual Basic How to Program",
35             "/images/small/vbhttp.jpg", "/images/large/vbhttp.jpg"));
36         books.add(new Book("Visual C# How to Program",
37             "/images/small/vcshttp.jpg", "/images/large/vcshttp.jpg"));
38         booksListView.setItems(books); // bind booksListView to books
39
40         // when ListView selection changes, show large cover in ImageView
41         booksListView.getSelectionModel().selectedItemProperty().
42             addListener(
43                 new ChangeListener<Book>() {
44                     @Override
45                     public void changed(ObservableValue<? extends Book> ov,
46                         Book oldValue, Book newValue) {
47                         coverImageView.setImage(
48                             new Image(newValue.getLargeImage()));
49                     }
50                 }
51             );
52     }
53 }

```

Fig. 13.15 | Controller for Cover Viewer application. (Part 2 of 2.)

@FXML Instance Variables

Lines 14–15 declare the controller's @FXML instance variables. Notice that `ListView` is a generic class. In this case, the `ListView` displays `Book` objects. Class `Book` contains three `String` instance variables with corresponding *set* and *get* methods:

- `title`—the book's title.
- `thumbImage`—the path to the book's thumbnail image (used in the next example).
- `largeImage`—the path to the book's large cover image.

The class also provides a `toString` method that returns the `Book`'s title and a constructor that initializes the three instance variables. You should copy class `Book` from this chapter's examples folder into the folder that contains `CoverViewer.fxml`, `CoverViewer.java` and `CoverViewerController.java`.

Instance Variable books

Lines 18–19 define the `books` instance variable as an `ObservableList<Book>` and initialize it by calling `FXCollections` static method `observableArrayList`. This method returns an empty collection object (similar to an `ArrayList`) that implements the `ObservableList` interface.

Initializing the books ObservableList

Lines 24–37 in method `initialize` create and add `Book` objects to the `books` collection. Line 38 passes this collection to `ListView` method `setItems`, which binds the `ListView` to the `ObservableList`. This *data binding* allows the `ListView` to display the `Book` objects automatically. By default, the `ListView` displays each `Book`'s `String` representation. (In the next example, you'll customize this.)

Listening for ListView Selection Changes

To synchronize the book cover that's being displayed with the currently selected book, we listen for changes to the `ListView`'s selected item. By default a `ListView` supports single selection—one item at a time may be selected. `ListsViews` also support multiple selection. The type of selection is managed by the `ListView`'s `MultipleSelectionModel` (a subclass of `SelectionMode` from package `javafx.scene.control`), which contains observable properties and various methods for manipulating the corresponding `ListView`'s items.

To respond to selection changes, you register a listener for the `MultipleSelectionModel`'s `selectedItem` property (lines 41–51). `ListView` method `getSelectionModel` returns a `MultipleSelectionModel` object. In this example, `MultipleSelectionModel`'s `selectedItemProperty` method returns a `ReadOnlyObjectProperty<Book>`, and the corresponding `ChangeListener` receives as its `oldValue` and `newValue` parameters the previously selected and newly selected `Book` objects, respectively.

Lines 47–48 use `newValue`'s large image path to initialize a new `Image` (package `javafx.scene.image`)—this loads the image from that path. We then pass the new `Image` to the `coverImageView`'s `setImage` method to display the `Image`.

13.6 Cover Viewer App: Customizing ListView Cells

In the preceding example, the `ListView` displayed a `Book`'s `String` representation (i.e., its title). In this example, you'll create a custom `ListView` cell factory to create cells that dis-

play each book as its thumbnail image and title using a VBox, an ImageView and a Label (Fig. 13.16).

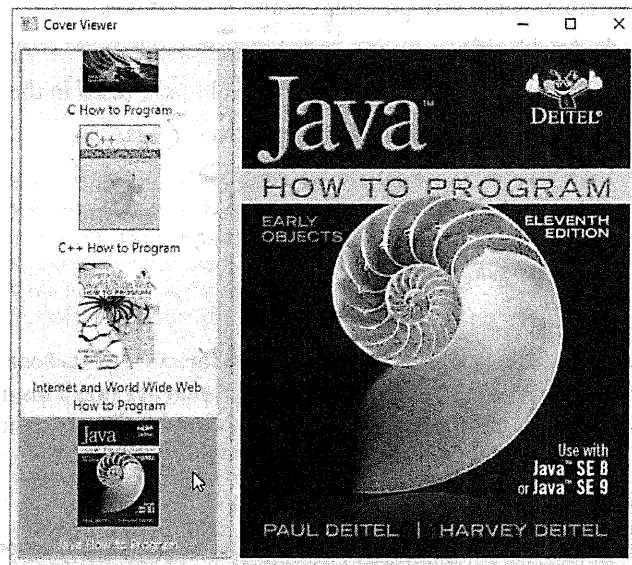


Fig. 13.16 | Cover Viewer app with *Java How to Program* selected.

13.6.1 Technologies Overview

ListCell Generic Class for Custom *ListView* Cell Formats

As you saw in Section 13.5, *ListView* cells display the *String* representations of a *ListView*'s items by default. To create a custom cell format, you must first define a subclass of the *ListCell* generic class (package `javafx.scene.control`) that specifies how to create a *ListView* cell. As the *ListView* displays items, it gets *ListCells* from its cell factory. You'll use the *ListView*'s **setCellFactory** method to replace the default cell factory with one that returns objects of the *ListCell* subclass. You'll override this class's **updateItem** method to specify the cells' custom layout and contents.

Programmatically Creating Layouts and Controls

So far, you've created GUIs visually using JavaFX Scene Builder. In this app, you'll also create a portion of the GUI programmatically—in fact, everything we've shown you in Scene Builder also can be accomplished in Java code directly. In particular, you'll create and configure a *VBox* layout containing an *ImageView* and a *Label*. The *VBox* represents the custom *ListView* cell format.

13.6.2 Copying the CoverViewer App

This app's FXML layout and classes *Book* and *CoverViewer* are identical to those in Section 13.5, and the *CoverViewerController* class has only one new statement. For this example, we'll show a new class that implements the custom *ListView* cell factory and the one new statement in class *CoverViewerController*. Rather than creating a new app from

scratch, copy the *CoverViewer* app from the previous example into a new folder named *CoverViewerCustomListView*.

13.6.3 ImageTextCell Custom Cell Factory Class

Class *ImageTextCell* (Fig. 13.17) defines the custom *ListView* cell layout for this version of the *Cover Viewer* app. The class extends *ListCell*<*Book*> because it defines a customized presentation of a *Book* in a *ListCell*.

```
1 // Fig. 13.16: ImageTextCell.java
2 // Custom ListView cell factory that displays an Image and text
3 import javafx.geometry.Pos;
4 import javafx.scene.control.Label;
5 import javafx.scene.control.ListCell;
6 import javafx.scene.image.Image;
7 import javafx.scene.image.ImageView;
8 import javafx.scene.layout.VBox;
9 import javafx.scene.text.TextAlignment;
10
11 public class ImageTextCell extends ListCell<Book> {
12     private VBox vbox = new VBox(8.0); // 8 points of gap between controls
13     private ImageView thumbImageView = new ImageView(); // initially empty
14     private Label label = new Label();
15
16     // constructor configures VBox, ImageView and Label
17     public ImageTextCell() {
18         vbox.setAlignment(Pos.CENTER); // center VBox contents horizontally
19
20         thumbImageView.setPreserveRatio(true);
21         thumbImageView.setFitHeight(100.0); // thumbnail 100 points tall
22         vbox.getChildren().add(thumbImageView); // attach to VBox
23
24         label.setWrapText(true); // wrap if text too wide to fit in label
25         label.setTextAlignment(TextAlignment.CENTER); // center text
26         vbox.getChildren().add(label); // attach to VBox
27
28         setPrefWidth(USE_PREF_SIZE); // use preferred size for cell width
29     }
30
31     // called to configure each custom ListView cell
32     @Override
33     protected void updateItem(Book item, boolean empty) {
34         // required to ensure that cell displays properly
35         super.updateItem(item, empty)
36
37         if (empty || item == null) {
38             setGraphic(null); // don't display anything
39         }
40         else {
41             // set ImageView's thumbnail image
42             thumbImageView.setImage(new Image(item.getThumbImage()));
43             label.setText(item.getTitle()); // configure Label's text
```

Fig. 13.17 | Custom *ListView* cell factory that displays an image and text.

```

44      setGraphic(vbox); // attach custom layout to ListView cell
45    }
46  }
47 }

```

Fig. 13.17 | Custom ListView cell factory that displays an image and text.

Constructor

The constructor (lines 17–29) configures the instance variables we use to build the custom presentation. Line 18 indicates that the VBox’s children should be centered. Lines 20–22 configure the ImageView and attach it to the VBox’s collection of children. Line 20 indicates that the ImageView should preserve the image’s aspect ratio, and line 21 indicates that the ImageView should be 100 points tall. Line 22 attaches the ImageView to the VBox.

Lines 24–26 configure the Label and attach it to the VBox’s collection of children. Line 24 indicates that the Label should wrap its text if its too wide to fit in the Label’s width, and line 25 indicates that the text should be centered in the Label. Line 26 attaches the Label to the VBox. Finally, line 28 indicates that the cell should use its preferred width, which is determined from the width of its parent ListView.

Method updateItem

Method updateItem (lines 32–46) configures the Label’s text and the ImageView’s Image then displays the custom presentation in the ListView. This method is called by the ListView’s cell factory when a ListView cell is required—that is, when the ListView is first displayed and when ListView cells are about to scroll onto the screen. The method receives the Book to display and a boolean indicating whether the cell that’s about to be created is empty. You must call the superclass’s version of updateItem (line 35) to ensure that the custom cells display correctly.

If the cell is empty or the item parameter is null, then there is no Book to display and line 38 calls the ImageTextCell’s inherited setGraphic method with null. This method receives as its argument the Node that should be displayed in the cell. Any JavaFX Node can be provided, giving you tremendous flexibility for customizing a cell’s appearance.

If there is a Book to display, lines 40–45 configure the ImageTextCell’s the Label and ImageView. Line 42 configures the Book’s Image and sets it to display in the ImageView. Line 43 sets the Label’s text to the Book’s title. Finally, line 38 uses method setGraphic to set the ImageTextCell’s VBox as the custom cell’s presentation.



Performance Tip 13.1

For the best ListView performance, it’s considered best practice to define the custom presentation’s controls as instance variables in the ListCell subclass and configure them in the subclass’s constructor. This minimizes the amount of work required in each call to method updateItem.

13.6.4 CoverViewerController Class

Once you’ve defined the custom cell layout, updating the CoverViewerController to use it requires that you set the ListView’s cell factory. Insert the following code as the last statement in the CoverViewerController’s initialize method:

```

booksListView.setCellFactory(
    new Callback<ListView<Book>, ListCell<Book>>() {
        @Override
        public ListCell<Book> call(ListView<Book> listView) {
            return new ImageTextCell();
        }
    });

```

and add an import for javafx.util.Callback.

The argument to ListView method setCellFactory is an implementation of the functional interface Callback (package javafx.util). This generic interface provides a call method that receives one argument and returns a value. In this case, we implement interface Callback with an object of an anonymous inner class. In Callback’s angle brackets the first type (ListView<Book>) is the parameter type for the interface’s call method and the second (ListCell<Book>) is the call method’s return type. The parameter represents the ListView in which the custom cells will appear. The call method call simply creates and returns an object of the ImageTextCell class.

Each time the ListView requires a new cell, the anonymous inner class’s call method will be invoked to get a new ImageTextCell. Then the ImageTextCell’s update method will be called to create the custom cell presentation. Note that by using a Java SE 8 lambda (Chapter 17) rather than an anonymous inner class, you can replace the entire statement that sets the cell factory with a single line of code.

8

13.7 Additional JavaFX Capabilities

This section overviews various additional JavaFX capabilities that are available in JavaFX 8 and JavaFX 9.

TableView Control

Section 13.5 demonstrated how to bind data to a ListView control. You often load such data from a database (Chapter 24, Accessing Databases with JDBC, and Chapter 29, Java Persistence API (JPA)). JavaFX’s TableView control (package javafx.scene.control) displays tabular data in rows and columns, and supports user interactions with that data.

Accessibility

In a Java SE 8 update, JavaFX added *accessibility* features to help people with visual impairments use their devices. For example, the screen readers in various operating systems can speak screen text or text that you provide to help users with visual impairments understand the purpose of a control. Visually impaired users must enable their operating systems’ screen-reading capabilities. JavaFX controls also support:

8

- GUI navigation via the keyboard—for example, the user can press the *Tab* key to jump from one control to the next. If a screen reader also is enabled, as the user moves the focus from control to control, the screen reader will speak appropriate information about each control (discussed below).
- A high-contrast mode to make controls more readable—as with screen readers, visually impaired users must enable this feature in their operating systems.

See your operating system's documentation for information on enabling its screen reader and high-contrast mode.

Every JavaFX Node subclass also has the following accessibility-related properties:

- `accessibleTextProperty`—A String that a screen reader speaks for a control. For example, a screen reader normally speaks the text displayed on a Button, but setting this property for a Button causes the screen reader to speak this property's text instead. You also can set this property to provide accessibility text for controls that do not have text, such as ImageViews.
- `accessibleHelpProperty`—A more detailed control description String than that provided by the `accessibleTextProperty`. This property's text should help the user understand the purpose of the control in the context of your app.
- `accessibleRoleProperty`—A value from the enum `AccessibleRole` (package `javafx.scene`). A screen reader uses this property value to determine the attributes and actions supported for a given control.
- `accessibleRoleDescriptionProperty`—A String text description of a control that a screen reader typically speaks followed by the control's contents (such as the text on a Button) or the value of the `accessibleTextProperty`.

In addition, you can add Labels to a GUI that describe other controls. In such cases, you should set each Label's `labelFor` property to the specific control the Label describes. For example, a TextField in which the user can enter a phone number might be preceded by a Label containing the text "Phone Number". If the Label's `labelFor` property references the TextField, then a screen reader will read the Label's text as well when describing the TextField to the user.

Third-Party JavaFX Libraries

JavaFX continues to become more popular. There are various open-source, third-party libraries, which define additional JavaFX capabilities that you can incorporate into your own apps. Some popular JavaFX libraries include:

- ControlsFX (<http://www.controlsfx.org/>) provides common dialogs, additional controls, validation capabilities, TextField enhancements, a SpreadsheetView, TableView enhancements and more. You can find the API documentation at <http://docs.controlsfx.org/> and various code samples at <http://code.controlsfx.org>. We use one of the open-source ControlsFX dialogs in Chapter 22.
- JFXtras (<http://jfxtras.org/>) also provides many additional JavaFX controls, including date/time pickers, controls for maintaining an agenda, a calendar control, additional window features and more.
- Medusa provides many JavaFX gauges that look like clocks, speedometers and more. You can view samples at <https://github.com/HanSolo/Medusa/blob/master/README.md>.

Creating Custom JavaFX Controls

You can create custom controls by extending existing JavaFX control classes to customize them or by extending JavaFX's `Control` class directly.

JavaFXPorts: JavaFX for Mobile and Embedded Devices

A key Java benefit is writing apps that can run on any device with a Java Virtual Machine (JVM), including notebook computers, desktop computers, servers, mobile devices and embedded devices (such as those used in the Internet of Things). Oracle officially supports JavaFX only for desktop apps. Gluon's open-source JavaFXPorts project brings the desktop version of JavaFX to mobile devices (iOS and Android) and devices like the inexpensive Raspberry Pi (<https://www.raspberrypi.org/>), which can be used as a standalone computer or for embedded-device applications. For more information on JavaFXPorts, visit

<http://javafxports.org/>

In addition, Gluon Mobile provides a mobile-optimized JavaFX implementation for iOS and Android. For more information, see

<http://gluonhq.com/products/mobile/>

Scenic View for Debugging JavaFX Scenes and Nodes

Scenic View is a debugging tool for JavaFX scenes and nodes. You embed Scenic View directly into your apps or run it as a standalone app. You can inspect your JavaFX scenes and nodes, and modify them dynamically to see how changes affect their presentation on the screen—without having to edit your code, recompile it and re-run it for each change. For more information, visit

<http://www.scenic-view.org>

JavaFX Resources and JavaFX in the Real World

Visit

<http://bit.ly/JavaFXResources>

for a lengthy and growing list of JavaFX resources that includes links to:

- articles
- tutorials (free and for purchase)
- key blogs and websites
- YouTube® videos
- books (for purchase)
- many libraries, tools, projects and frameworks
- slide shows from JavaFX presentations and
- various real-world examples of JavaFX in use.

13.8 JavaFX 9: Java SE 9 JavaFX Updates

This section overviews several JavaFX 9 changes and enhancements.

Java SE 9 Modularization

Java SE 9's biggest new software-engineering feature is the module system. This applies to JavaFX 9 as well. The key JavaFX 9 modules are:

- `javafx.base`—Contains the packages required by all JavaFX 9 apps. All the other JavaFX 9 modules depend on this one.
- `javafx.controls`—Contains the packages for controls, layouts and charts, including the various controls we demonstrated in this chapter and Chapter 12.
- `javafx.fxml`—Contains the packages for working with FXML, including the FXML features we demonstrated in this chapter and Chapter 12.
- `javafx.graphics`—Contains the packages for working with graphics, animation, CSS (for styling nodes), text and more (Chapter 22, JavaFX Graphics, Animation and Video).
- `javafx.media`—Contains the packages for incorporating audio and video (Chapter 22, JavaFX Graphics, Animation and Video).
- `javafx.swing`—Contains the packages for integrating into JavaFX 9 apps Swing GUI components (Chapter 26, Swing GUI Components: Part 1, and Chapter 35, Swing GUI Components: Part 2).
- `javafx.web`—Contains the package for integrating web content.

In your apps, if you use modularization and JDK 9, only the modules required by your app will be loaded at runtime. Otherwise, your app will continue to work as it did previously, provided that you did not use so-called internal APIs—that is, undocumented Java APIs that are not meant for public use. In the modularized JDK 9, such APIs are automatically *private* and inaccessible to your apps—any code that depends on pre-Java-SE-9 internal APIs will not compile. We discuss modularization in more detail in our online Java SE 9 treatment. See the Preface for details.

9 New Public Skinning APIs

In Chapter 22, JavaFX Graphics, Animation and Video, we demonstrate how to format JavaFX objects using a technology called *Cascading Style Sheets (CSS)* that was originally developed for styling the elements in web pages. As you'll see, CSS allows you to specify *presentation* (e.g., fonts, spacing, sizes, colors, positioning) separately from the GUI's *structure* and *content* (layout containers, shapes, text, GUI components, etc.). If a JavaFX GUI's presentation is determined entirely by a style sheet (which specifies the rules for styling the GUI), you can simply swap in a new style sheet—sometimes called a *skin*—to change the GUI's appearance. This is commonly called **skinning**.

Each JavaFX control also has a skin class that determines its default appearance. In JavaFX 8, skin classes are defined as internal APIs, but many developers create custom skins by extending these skin classes. In JavaFX 9, the skin classes are now public APIs in the package `javafx.scene.control.skin`. You can extend the appropriate skin class to customize the look-and-feel for a given type of control. You then create an object of your custom skin class and set it for a control via its `setSkin` method.

9 GTK+ 3 Support on Linux

GTK+ (GIMP Toolkit—<http://gtk.org>) is a GUI toolkit that JavaFX uses behind the scenes to render GUIs and graphics on Linux. In Java SE 9, JavaFX now supports GTK+ 3—the latest version of GTK+.

High-DPI Screen Support

In a Java SE 8 update, JavaFX added support for High DPI (dots-per-inch) screens on Windows and macOS. Java SE 9 adds Linux High-DPI support, as well as capabilities to programmatically manipulate the scale at which JavaFX apps are rendered on Windows, macOS and Linux.

Updated GStreamer

JavaFX implements its audio and video multimedia capabilities using the open-source GStreamer framework (<https://gstreamer.freedesktop.org>). JavaFX 9 incorporates a more recent version of GStreamer with various bug fixes and performance enhancements.

Updated WebKit

JavaFX's WebView control enables you to embed web content in your JavaFX apps. WebView is based on the open source WebKit framework (<http://www.webkit.org>)—a web browser engine that supports loading and rendering web pages. JavaFX 9 incorporates an updated version of WebKit.

13.9 Wrap-Up

In this chapter, we continued our presentation of JavaFX. We discussed JavaFX layout panes in more detail and used `BorderPane`, `TitledPane` and `Pane` to arrange controls.

You learned about the many mouse events supported by JavaFX nodes, and we used the `onMouseDragged` event in a simple **Painter** app that displayed `Circles` as the user dragged the mouse across an `Pane`. The **Painter** app allowed the user to choose the current color and pen size from groups of mutually exclusive `RadioButtons`. You used `ToggleGroups` to manage the relationship between the `RadioButtons` in each group. You also learned how to provide a so-called user data Object for a control. When a `RadioButton` was selected, you obtained it from the `ToggleGroup`, then accessed the `RadioButton`'s user data Object to determine the drawing color or pen size.

We discussed property binding and property listeners, then used them to implement a **Color Chooser** app. You bound a `TextField`'s text to a `Slider`'s value to automatically update the `TextField` when the user moved the `Slider`'s thumb. You also used a property listener to allow the app's controller to update the color of a `Rectangle` when a `Slider`'s value changed.

In our **Cover Viewer** app, we showed how to bind an `ObservableList` collection to a `ListView` control to populate it with the collection's elements. By default, each object in the collection was displayed as a `String` in the `ListView`. You configured a property listener to display an image in an `ImageView` when the user selected an item in the `ListView`. We modified the **Cover Viewer** app to use a custom `ListView` cell factory to specify the exact layout of a `ListView` cell's contents. Finally, we introduced several other JavaFX capabilities and the Java SE 9 changes to JavaFX.

In the next chapter, we discuss class `String` and its methods. We introduce regular expressions for pattern matching in strings and demonstrate how to validate user input with regular expressions.

Summary

Section 13.2 Laying Out Nodes in a Scene Graph

- A layout determines the size and positioning of nodes in the scene graph.
- In general, a node's size should not be defined explicitly.
- In addition to the width and height properties associated with every JavaFX node, most JavaFX nodes have the properties `prefWidth`, `prefHeight`, `minWidth`, `minHeight`, `maxWidth` and `maxHeight` that specify a node's *range* of acceptable sizes as it's laid out within its parent node.
- The minimum size properties specify a node's smallest allowed size in points.
- The maximum size properties specify a node's largest allowed size in points.
- The preferred size properties specify a node's preferred width and height that should be used by a layout in most cases.
- A node's position should be defined relative to its parent node and the other nodes in its parent.
- Layout panes are container nodes that arrange their child nodes in a scene graph relative to one another, based on their sizes and positions.
- Most JavaFX layout panes use relative positioning.

Section 13.3.1 Technologies Overview

- RadioButtons function as mutually exclusive options.
- You add multiple RadioButtons to a `ToggleGroup` to ensure that only one RadioButton in a given group is selected at a time.
- If you programmatically create a `ToggleGroup` (rather than declaring it in FXML), you can call `RadioButton`'s `setToggleGroup` method to specify its `ToggleGroup`.
- A `BorderPane` layout container arranges controls into one or more of five regions—top, right, bottom, left and center. The top and bottom areas have the same width as the `BorderPane`. The left, center and right areas fill the vertical space between the top and bottom areas. Each area may contain only one control or one layout container that, in turn, may contain other controls.
- All the areas in a `BorderPane` are optional: If the top or bottom area is empty, the left, center and right areas expand vertically to fill that area. If the left or right area is empty, the center expands horizontally to fill that area.
- A `TitledPane` displays a title at its top and is a collapsible panel containing a layout node, which in turn contains other nodes.
- The `javafx.scene.shape` package contains various classes for creating 2D and 3D shape nodes that can be displayed in a scene graph.
- Nodes are attached to an `Pane` layout at a specified *x-y* coordinate measured from the `Pane`'s upper-left corner.
- JavaFX nodes support various mouse events.
- JavaFX supports other types of input events, such as touch-oriented events and key events.
- Each JavaFX control has a `setUserData` method that receives an `Object`. You can use this to store any object you'd like to associate with that control—typically this `Object` is used when responding to the control's events.

Section 13.3.2 Creating the Painter.fxml File

- If you already have an FXML file open in Scene Builder, you can choose **File > New** to create a new FXML file, then save it.

Section 13.3.3 Building the GUI

- A `VBox`'s `Spacing` property specifies vertical spacing between its controls.
- Setting a node's `Max Height` property to `MAX_VALUE` enables the node to occupy the full height of its parent node.
- The `Style -fx-background-color` specifies a node's background color.
- A `TitledPane`'s `Text` property specifies the title at the top of the `TitledPane`.
- A `RadioButton`'s `Text` property specifies the text that appears next to the `RadioButton`.
- A `RadioButton`'s `Selected` property specifies whether the `RadioButton` is selected.
- Setting a `RadioButton`'s `Toggle Group` property in FXML adds the `RadioButton` to that `ToggleGroup`.
- Setting a control's `Max Width` property to `MAX_VALUE` enables the control to fill its parent node's width.
- A control's `On Mouse Dragged` event handler (located under the **Mouse** heading in the **Code** section) specifies what to do when the user drags the mouse on the control.
- To specify what to do when a user interacts with a `RadioButton`, set its `On Action` event handler.
- To specify what to do when a user interacts with a `Button`, set its `On Action` event handler.

Section 13.3.5 PainterController Class

- Top-level types are not declared inside another type.
- Java allows you to declare classes, interfaces and enums inside other classes—these are called nested types.
- Each mouse event handler you define must provide one parameter of type `MouseEvent` (package `javafx.scene.input`). When the event occurs, this parameter contains information about the event, such as its location, whether any mouse buttons were pressed, which node the user interacted with and more.
- Each layout pane has a `getChildren` method that returns an `ObservableList<Node>` collection containing the layout's child nodes. An `ObservableList` provides methods for adding and removing elements.
- All JavaFX shapes inherit indirectly from class `Node` in the `javafx.scene` package.
- `ToggleGroup` method `getSelectedToggle` returns the `Toggle` that's currently selected. Class `RadioButton` is one of several controls (others are `RadioButtonMenuItem` and `ToggleButton`) that implements interface `Toggle`.
- `Toggle`'s `getUserData` method gets the user data `Object` that's associated with a control.

Section 13.4.1 Technologies Overview

- In the `RGBA` color system, every color is represented by its red, green and blue color values, each ranging from 0 to 255, where 0 denotes no color and 255 full color. The alpha value (*A*)—which ranges from 0.0 to 1.0—represents a color's opacity, with 0.0 being completely transparent and 1.0 completely opaque.
- A property is defined by creating *set* and *get* methods with specific naming conventions. Typically, such methods manipulate a corresponding private instance variable that has the same name as the property, but this is not required. If the property represents a boolean value, its *get* method name typically begins with "is" rather than "get."
- JavaFX properties are observable—when a property's value changes, other objects can respond accordingly.

Summary

Section 13.2 Laying Out Nodes in a Scene Graph

- A layout determines the size and positioning of nodes in the scene graph.
- In general, a node's size should not be defined explicitly.
- In addition to the width and height properties associated with every JavaFX node, most JavaFX nodes have the properties `prefWidth`, `prefHeight`, `minWidth`, `minHeight`, `maxWidth` and `maxHeight` that specify a node's *range* of acceptable sizes as it's laid out within its parent node.
- The minimum size properties specify a node's smallest allowed size in points.
- The maximum size properties specify a node's largest allowed size in points.
- The preferred size properties specify a node's preferred width and height that should be used by a layout in most cases.
- A node's position should be defined relative to its parent node and the other nodes in its parent.
- Layout panes are container nodes that arrange their child nodes in a scene graph relative to one another, based on their sizes and positions.
- Most JavaFX layout panes use relative positioning.

Section 13.3.1 Technologies Overview

- RadioButtons function as mutually exclusive options.
- You add multiple RadioButtons to a ToggleGroup to ensure that only one RadioButton in a given group is selected at a time.
- If you programmatically create a ToggleGroup (rather than declaring it in FXML), you can call RadioButton's `setToggleGroup` method to specify its ToggleGroup.
- A BorderPane layout container arranges controls into one or more of five regions—top, right, bottom, left and center. The top and bottom areas have the same width as the BorderPane. The left, center and right areas fill the vertical space between the top and bottom areas. Each area may contain only one control or one layout container that, in turn, may contain other controls.
- All the areas in a BorderPane are optional: If the top or bottom area is empty, the left, center and right areas expand vertically to fill that area. If the left or right area is empty, the center expands horizontally to fill that area.
- A TitledPane displays a title at its top and is a collapsible panel containing a layout node, which in turn contains other nodes.
- The `javafx.scene.shape` package contains various classes for creating 2D and 3D shape nodes that can be displayed in a scene graph.
- Nodes are attached to an Pane layout at a specified *x-y* coordinate measured from the Pane's upper-left corner.
- JavaFX nodes support various mouse events.
- JavaFX supports other types of input events, such as touch-oriented events and key events.
- Each JavaFX control has a `setUserData` method that receives an Object. You can use this to store any object you'd like to associate with that control—typically this Object is used when responding to the control's events.

Section 13.3.2 Creating the Painter.fxml File

- If you already have an FXML file open in Scene Builder, you can choose **File > New** to create a new FXML file, then save it.

Section 13.3.3 Building the GUI

- A VBox's `Spacing` property specifies vertical spacing between its controls.
- Setting a node's `Max Height` property to `MAX_VALUE` enables the node to occupy the full height of its parent node.
- The `Style -fx-background-color` specifies a node's background color.
- A TitledPane's `Text` property specifies the title at the top of the TitledPane.
- A RadioButton's `Text` property specifies the text that appears next to the RadioButton.
- A RadioButton's `Selected` property specifies whether the RadioButton is selected.
- Setting a RadioButton's `Toggle Group` property in FXML adds the RadioButton to that ToggleGroup.
- Setting a control's `Max Width` property to `MAX_VALUE` enables the control to fill its parent node's width.
- A control's `On Mouse Dragged` event handler (located under the **Mouse** heading in the **Code** section) specifies what to do when the user drags the mouse on the control.
- To specify what to do when a user interacts with a RadioButton, set its `On Action` event handler.
- To specify what to do when a user interacts with a Button, set its `On Action` event handler.

Section 13.3.5 PainterController Class

- Top-level types are not declared inside another type.
- Java allows you to declare classes, interfaces and enums inside other classes—these are called nested types.
- Each mouse event handler you define must provide one parameter of type `MouseEvent` (package `javafx.scene.input`). When the event occurs, this parameter contains information about the event, such as its location, whether any mouse buttons were pressed, which node the user interacted with and more.
- Each layout pane has a `getChildren` method that returns an `ObservableList<Node>` collection containing the layout's child nodes. An `ObservableList` provides methods for adding and removing elements.
- All JavaFX shapes inherit indirectly from class `Node` in the `javafx.scene` package.
- `ToggleGroup` method `getSelectedToggle` returns the `Toggle` that's currently selected. Class `RadioButton` is one of several controls (others are `RadioButtonMenuItem` and `ToggleButton`) that implements interface `Toggle`.
- `Toggle`'s `getUserData` method gets the user data Object that's associated with a control.

Section 13.4.1 Technologies Overview

- In the RGBA color system, every color is represented by its red, green and blue color values, each ranging from 0 to 255, where 0 denotes no color and 255 full color. The alpha value (A)—which ranges from 0.0 to 1.0—represents a color's opacity, with 0.0 being completely transparent and 1.0 completely opaque.
- A property is defined by creating *set* and *get* methods with specific naming conventions. Typically, such methods manipulate a corresponding private instance variable that has the same name as the property, but this is not required. If the property represents a boolean value, its *get* method name typically begins with "is" rather than "get."
- JavaFX properties are observable—when a property's value changes, other objects can respond accordingly.

- One way to respond to a property change is via a property binding, which enables a property of one object to be updated when a property of another object changes.
- Property bindings are not limited to JavaFX controls. Package `javafx.beans.property` contains many classes that you can use to define bindable properties in your own classes.
- A property listener is an event handler that's invoked when a property's value changes. In the event handler, you can respond to the property change in a manner appropriate for your app.

Section 13.4.2 Building the GUI

- `Circle` and `Rectangle` are located in the Scene Builder Library's **Shapes** section.

Section 13.4.4 ColorChooserController Class

- A controller class's `initialize` method often configures property bindings and property listeners.
- Each `TextField` has a `text` property that's returned by its `textProperty` method as a `StringProperty` (package `javafx.beans.property`).
- `StringProperty` method `bind` receives an `ObservableValue` as an argument. When the `ObservableValue` changes, the bound property is updated accordingly.
- `Slider` method `valueProperty` returns a `Slider`'s value property as an object of class `DoubleProperty`—an observable double value.
- `DoubleProperty` method `asString` returns a `StringBinding` object (which is an `ObservableValue`) that produces a `String` representation of the `DoubleProperty`.
- To perform an arbitrary task when a property's value changes, you can register a property listener.

Section 13.5 Cover Viewer App: Data-Driven GUIs with JavaFX Collections

- JavaFX provides a comprehensive model for allowing GUIs to interact with data.

Section 13.5.1 Technologies Overview

- A `ListView` control displays a collection of objects.
- Though you can individually add items to a `ListView`, you'll often bind an `ObservableList` object to the `ListView`.
- If you make changes to an `ObservableList`, its observer (such as a `ListView`) will automatically be notified of those changes.
- Package `javafx.collections` defines `ObservableList` (similar to an `ArrayList`) and other observable collection interfaces.
- Class `FXCollections` provides static methods for creating and manipulating observable collections.

Section 13.5.5 CoverViewerController Class

- `FXCollections` static method `observableArrayList` returns an empty collection object (similar to an `ArrayList`) that implements the `ObservableList` interface.
- `ListView` method `setItems` receives an `ObservableList` and binds the `ListView` to it. This data binding allows the `ListView` to display the `ObservableList`'s objects automatically—as `Strings` by default.
- By default a `ListView` supports single selection—one item at a time may be selected. `Lists` also support multiple selection. The type of selection is managed by the `ListView`'s `SelectionMode` (a subclass of `SelectionMode` from package `javafx.scene.control`), which contains observable properties and various methods for manipulating the corresponding `ListView`'s items.

- To respond to selection changes, register a listener for the `SelectionMode`'s `selectedItem` property.
- `ListView` method `getSelectionMode` returns a `SelectionMode` object.
- `SelectionMode`'s `selectedItemProperty` method returns a `ReadOnlyObjectProperty`, and the corresponding `ChangeListener` receives as its `oldValue` and `newValue` parameters the previously selected and newly selected objects, respectively.

Section 13.6.1 Technologies Overview

- To create a custom `ListView` cell format, you must first define a subclass of the `ListCell` generic class (package `javafx.scene.control`) that specifies how to create a `ListView` cell.
- As the `ListView` displays items, it gets `ListView` cells from its cell factory.
- You'll use the `ListView`'s `setCellFactory` method to replace the default cell factory with one that returns objects of the `ListCell` subclass. You override this class's `updateItem` method to specify the cells' custom layout and contents.
- Everything you can do in Scene Builder also can be accomplished in Java code.

Section 13.6.3 ImageTextCell Custom Cell Factory Class

- A custom `ListView` cell layout is defined as a subclass of `ListCell<Type>`, where *Type* is the type of the object displayed in a `ListView` cell.
- The `ListCell<Type>` subclass's `updateItem` method creates the custom presentation. This method is called by the `ListView`'s cell factory when a `ListView` cell is required—that is, when the `ListView` is first displayed and when `ListView` cells are about to scroll onto the screen.
- Method `updateItem` receives the object to display and a `boolean` indicating whether the cell that's about to be created is empty. You must call the superclass's version of `updateItem` to ensure that the custom cells display correctly.
- `ListCell<Type>`'s `setGraphic` method receives a JavaFX `Node` representing the customized cell's appearance.

Section 13.6.4 CoverViewerController Class

- Once you've defined the custom cell layout, you must set the `ListView`'s cell factory.
- The argument to `ListView` method `setCellFactory` is an implementation of interface `Callback` (package `javafx.util`). This generic interface provides a `call` method that receives one argument and returns an object of the custom `ListCell<Type>` subclass.

Section 13.7 Additional JavaFX Capabilities

- JavaFX's `TableView` control (package `javafx.scene.control`) displays tabular data in rows and columns, and supports user interactions with that data.
- In a Java SE 8 update, JavaFX added accessibility features to help people with visual impairments use their devices. These features include screen-reader support, GUI navigation via the keyboard and a high-contrast mode to make controls more readable.
- See your operating system's documentation for information on enabling its screen reader and high-contrast mode.
- Every JavaFX `Node` subclass also has accessibility-related properties.
- The `accessibleTextProperty` is a `String` that a screen reader speaks for a control.
- The `accessibleHelpProperty` is a more detailed control description `String` than that provided by the `accessibleTextProperty`. This property's text should help the user understand the purpose of the control in the context of your app.

- The `accessibleRoleProperty` is a value from the enum `AccessibleRole` (package `javafx.scene`). A screen reader uses this property value to determine the attributes and actions supported for a given control.
- The `accessibleRoleDescriptionProperty` is a `String` text description of a control that a screen reader typically speaks followed by the control's contents or the value of the `accessibleTextProperty`.
- You can add `Labels` to a GUI that describe other controls. In such cases, you should set each `Label`'s `labelFor` property to the specific control the `Label` describes. If a `Label`'s `labelFor` property references another control, a screen reader will read the `Label`'s text when describing the control to the user.
- You can create custom controls by extending existing JavaFX control classes to customize them or by extending JavaFX's `Control` class directly.
- **Scenic View** (<http://www.scenic-view.org/>) is a debugging tool for JavaFX scenes and nodes. You embed **Scenic View** directly into your apps or run it as a standalone app. You can inspect your JavaFX scenes and nodes, and modify them dynamically to see how changes affect their presentation on the screen—without having to edit your code, recompile it and re-run it for each change.

Section 13.8 JavaFX 9: Java SE 9 JavaFX Updates

- JavaFX 9's biggest new feature is modularization.
- In your apps, if you use modularization and JDK 9, only the modules required by your app will be loaded at runtime. Otherwise, your app will continue to work as it did previously, provided that you did not use so-called internal APIs—that is, undocumented Java APIs that are not meant for public use.
- If a JavaFX GUI's presentation is determined entirely by a style sheet (which specifies the rules for styling the GUI), you can simply swap in a new style sheet—sometimes called a skin—to change the GUI's appearance. This is commonly called skinning.
- Each JavaFX control also has a skin class that determines its default appearance.
- In JavaFX 8, skin classes are defined as internal APIs. In JavaFX 9, the skin classes are now public APIs in the package `javafx.scene.control.skin`.
- You can extend the appropriate skin class to customize the look-and-feel for a given type of control. You then create an object of your custom skin class and set it for a control via its `setSkin` method.
- JavaFX 9 supports GTK+ 3—the latest version of GTK+.
- In a Java SE 8 update, JavaFX added support for High-DPI (dots-per-inch) screens on Windows and macOS. Java SE 9 adds Linux High-DPI support.
- JavaFX 9 adds features to programmatically manipulate the scale at which JavaFX apps are rendered on Windows, macOS and Linux.
- JavaFX implements its audio and video multimedia capabilities using the open-source GStreamer framework (<https://gstreamer.freedesktop.org>). JavaFX 9 incorporates a more recent version of GStreamer with various bug fixes and performance enhancements.
- JavaFX's `WebView` control enables you to embed web content in your JavaFX apps. `WebView` is based on the open source WebKit framework (<http://www.webkit.org>)—a web browser engine that supports loading and rendering web pages. JavaFX 9 incorporates an updated version of WebKit.

Self-Review Exercises

13.1 Fill in the blanks in each of the following:

- You add multiple `RadioButtons` to a(n) _____ to ensure that only one `RadioButton` in a given group is selected at a time.
- A(n) _____ determines the size and positioning of nodes in the scene graph.
- A(n) _____ displays a title at its top and is a collapsible panel containing a layout node, which in turn contains other nodes.
- The _____ package contains various classes for creating 2D and 3D shape nodes that can be displayed in a scene graph.
- A control's _____ event handler specifies what to do when the user drags the mouse on the control.
- To specify what to do when a user interacts with a `RadioButton`, set its event handler.
- Each `TextField` has a text property that's returned by its `textProperty` method as a(n) _____ (package `javafx.beans.property`).
- `DoubleProperty` method `asString` returns a(n) _____ object (which is an `ObservableValue`) that produces a `String` representation of the `DoubleProperty`.
- Though you can individually add `ListView` items you'll often bind a(n) _____ object to the `ListView`.
- Class _____ provides static methods for creating and manipulating observable collections.
- As the `ListView` displays items, it gets `ListView` cells from its _____.
- JavaFX's _____ control enables you to embed web content in your JavaFX apps.
- In a Java SE 8 update, JavaFX added _____ to help people with visual impairments use their devices. These features include screen-reader support, GUI navigation via the keyboard and a high-contrast mode to make controls more readable.

13.2 State whether each of the following is *true* or *false*. If *false*, explain why:

- In general, a node's size should be defined explicitly.
- A node's position should be defined relative to its parent node and the other nodes in its parent.
- Most JavaFX layout panes use fixed positioning.
- `RadioButtons` function as mutually exclusive options.
- Layout panes are container nodes that arrange their child nodes in a scene graph relative to one another, based on their sizes and positions.
- `ToggleGroup` method `getToggle` returns the `Toggle` that's currently selected.
- Each layout pane has a `getChildren` method that returns an `ObservableList<Node>` collection containing the layout's child nodes.
- The only way to respond to a property change is via a property binding, which enables a property of one object to be updated when a property of another object changes.
- JavaFX properties are observable—when a property's value changes, other objects can respond accordingly.
- Property bindings are limited to JavaFX controls.
- In the RGBA color system, every color is represented by its red, green and blue color values, each ranging from 0 to 255, where 0 denotes no color and 255 full color. The alpha value (A)—which ranges from 0.0 to 1.0—represents a color's opacity, with 0.0 being completely transparent and 1.0 completely opaque.
- To respond to `ListView` selection changes, you register a listener for the `SingleSelectionModel`'s `selectedItem` property.

- m) To create a custom `ListView` cell format, you must first define a subclass of the `CellFormat` generic class (package `javafx.scene.control`) that specifies how to create a `ListView` cell.
- n) You can build JavaFX GUIs only by using Scene Builder.
- o) JavaFX's `TableView` control (package `javafx.scene.control`) displays tabular data in rows and columns, and supports user interactions with that data.
- p) You can add `Labels` to a GUI that describe other controls. In such cases, you should set each `Label`'s `describesControl` property to the specific control the `Label` describes.

Answers to Self-Review Exercises

13.1 a) `ToggleGroup`. b) `layout`. c) `TitledPane`. d) `javafx.scene.shape`. e) `On Mouse Dragged`. f) `On Action`. g) `StringProperty`. h) `StringBinding`. i) `ObservableList`. j) `FXCollections`. k) `cell factory`. l) `WebView`. m) accessibility features.

13.2 a) False. In general, a node's size shouldn't be defined explicitly. b) True. c) False. Most JavaFX layout panes use relative positioning. d) True. e) True. f) False. `ToggleGroup` method `getSelectedToggle` returns the `Toggle` that's currently selected. g) True. h) False. You can also respond to property changes with a property listener, which is an event handler that's invoked when a property's value changes. i) True. j) False. Property bindings are not limited to JavaFX controls. Package `javafx.beans.property` contains many classes that you can use to define bindable properties in your own classes. k) True. l) False. To respond to `ListView` selection changes, you register a listener for the `MultipleSelectionModel`'s `selectedItem` property. m) False. To create a custom `ListView` cell format, you must first define a subclass of the `ListCell` generic class (package `javafx.scene.control`) that specifies how to create a `ListView` cell. n) False. Everything you can do in Scene Builder also can be accomplished in Java code. o) True. p) You can add `Labels` to a GUI that describe other controls. In such cases, you should set each `Label`'s `labelFor` property to the specific control the `Label` describes.

Exercises

13.3 (*Painter App Modification*) Incorporate the RGBA color chooser you created in the `Color Chooser` app (Section 13.4) into the `Painter` app (Section 13.3) so that the user can choose any drawing color. Changing a `Slider`'s value should update the color swatch displayed to the user and set the `brushColor` instance variable to the current `Color`.

13.4 (*Contacts App*) Create a `Contacts` app modeled after the `Cover Viewer` app (Sections 13.5—13.6). Store the contact information in an `ObservableList` of `Contact` objects. A `Contact` should contain first name, last name, email and phone number properties (you can provide others). When the user selects a contact from the contacts list, its information should display in a `Grid` of `TextFields`. As the information is modified (a `Contact`'s data is updated, a new `Contact` is added or an existing `Contact` is deleted), the contacts `ListView` should display the updates. The `ListView` should display the `Contact`'s last names.

13.5 (*Contacts App Modification*) Modify the `Contacts` app from the preceding exercises to include an image for each `Contact`. Provide a custom `ListView` cell factory that displays the `Contact`'s full name and picture with the names in sorted order by last name.

13.6 (*Tip Calculator Modification*) The `Tip Calculator` app from Section 12.5 does not need a `Button` to perform its calculations. Reimplement this app to use property listeners to perform the calculations whenever the user modifies the bill amount or changes the custom tip percentage. Also use a property binding to update the `Label` that displays the tip percentage.

13.7 (*Advanced Project: Color Chooser App Modification*) The property bindings we created in the `Color Chooser` app (Section 13.4) allowed a `TextField`'s text to update when a `Slider`'s value changed, but not vice versa. JavaFX also supports bi-directional property bindings. Research bi-directional property bindings online, then create bidirectional bindings between the `Sliders` and the `TextFields` such that modifying a `TextField`'s value updates the corresponding slider.

13.8 (*Project: Mini Web Browser*) Investigate the capabilities of JavaFX's `WebView` control and `WebEngine` class, then create a JavaFX app that provides basic web browsing capabilities. For an introduction to these classes visit <https://docs.oracle.com/javase/8/javafx/embedded-browser-tutorial/overview.htm>.

(Optional) GUI and Graphics Case Study Exercise: Interactive Polymorphic Drawing App

13.9 (*Project: Interactive Polymorphic Drawing App*) Implement a JavaFX app that uses the `MyShape` hierarchy from the GUI and Graphics Case Study (on the book's Companion Website) to create a fully interactive drawing application. The classes of the `MyShape` hierarchy require no additional changes. Your app should store the shapes in an `ArrayList<MyShape>`. The user should be able to specify drawing characteristics such as which shape to draw, the stroke color, the fill color and whether the shape should be hollow or filled. Provide mouse event handling to enable the user to position and size each shape by dragging the mouse. Also enable the user to undo the last shape drawn and to clear the entire drawing.